VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

A PROJECT REPORT

On

"Smart Air Quality Monitoring System Using IOT and AI"

A dissertation submitted in partial fulfillment of the requirement for the degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

AMITH C PATIL	**(1RG23CS006)**
B NITHYA SHREE	**(1RG23CS012)**
NIHAR T N	**(1RG23CS043)**
VARUN P S	**(1RG23CS078)**

Under The Guidance of

Mrs. Bhagyashri Wakde

Assistant Professor

Dept. of CSE, Bengaluru-32

Department of Computer Science & Engineering

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

Cholanagar, R. T. Nagar Post, Bengaluru-560032

2026-2027

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University)

Cholanagar, R.T. Nagar Post, Bengaluru-560032

Department of Computer Science & Engineering

CERTIFICATE

Phase - II

This is to certify that the Project Report entitled **"Smart Air Quality Monitoring System Using IOT and AI"** is a Bonafide work carried out by **Mr. AMITH C PATIL (1RG23CS006), Mr. NIHAR T N(1RG23CS043), Mrs. B NITHYA SHREE (1RG23CS012), Mr. VARUN P S (1RG23CS078) in** partial fulfillment for the award of **Bachelor of Engineering** in **Computer Science and Engineering** under **Visvesvaraya Technological University, Belagavi,** during the year **2026-2027.** It is certified that all corrections/suggestions given for Internal Assessment have been incorporated in the report. This report has been approved as it satisfies the academic requirements in respect of project work (BCS786) prescribed for the said degree.

Signature of guide	Signature of HOD	Signature of Principal
Mrs.Bhagyashri Wakde	**Dr. Arudra A**	**Dr. D G Anand**
Assistant Professor,	Associate Professor, HOD	Principal, RGIT,
Dept. of CSE	Dept. of CSE	Benagluru-32
RGIT, Bengaluru-32	RGIT, Bengaluru-32	

External Viva

Name of the Examiners	**Signature with date**
1.	
2.	

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018

RAJIV GANDHI INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

We hereby declare that the project work entitled **"Smart Air Quality Monitoring System Using IOT and AI"** submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2026-2027**, is record of an original work done by us under the guidance of **Mrs. Bhagyashri Wakde,** Assistant Professor, Rajiv Gandhi Institute of Technology, Bengaluru and this project work is submitted in the partial fulfillment of requirements for the award of the degree of **Bachelor of Engineering** in **Computer Science & Engineering.** The results embodied in this report have not been submitted to any other University or Institute for award of any degree or diploma.

AMITH C PATIL (1RG23CS006)

B NITHYA SHREE (1RG23CS012)

NIHAR T N (1RG23CS043)

VARUN P S (1RG23CS078)

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude and respect to the **Rajiv Gandhi Institute of Technology, Bengaluru** for providing us an opportunity to carry out our project work. We express our sincere regards and thanks to **Dr. D. G ANAND**, Principal, RGIT, Bengaluru.

We would like to thanks **Dr. ARUDRA A**, Associate Professor and HOD, Department of Computer Science & Engineering, RGIT, Bengaluru, for their encouragement and support throughout the Project

With a profound sense of gratitude, we acknowledge the guidance and support extended towards us by project coordinators **Mrs.BHAGYASHRI WAKDE**, Assistant Professor, Department of CSE, RGIT, Bengaluru. **Mrs. SONIYA KOMAL V**, Assistant Professor, Department of CSE, RGIT, Bengaluru. Their incessant encouragement and valuable technical support have been of immense help in realizing this project.

We extend our heartfelt gratitude to, **Mrs.BHAGYASHRI WAKDE**, Assistant Professor, Department of Computer Science & Engineering, RGIT, Bengaluru, for their unwavering support and guidance in our project. Their expertise has been instrumental in shaping its success.

We also extend our thanks to the entire faculty of the Department of CSE, RGIT, Bengaluru, who have encouraged throughout the course of Bachelor Degree. We extend our heartfelt thanks to our family for their full support and unwavering encouragement, which provided us with a safe and conducive environment for the completion of this project. Their assistance and belief in our abilities have been instrumental in this achievement.

AMITH C PATIL (1RG23CS006)

B NITHYA SHREE (1RG23CS012)

NIHAR T N (1RG23CS043)

VARUN P S (1RG23CS078)

ABSTRACT

The rapid convergence of the Internet of Things (IoT), low-cost microcontrollers, and cloud analytics has transformed real-time environmental monitoring. This project presents an AI-Integrated Smart Environment Monitoring System (SEMS) that uses an ESP32-WROOM-32 with seven sensors (DHT22, BH1750, DSM501A, MQ-135, MQ-7, PMS5003, and Neo-6M GPS) to track indoor environment quality. Data is sent via MQTT over Wi-Fi to Firebase Realtime Database and ThingSpeak, and multi-stage calibration achieved $R^2 > 0.99$ and RMSE < 0.5 across all core parameters. An on-device TensorFlow Lite advisory module runs inference in 7–18 ms and reaches 91.4% expert-rated suggestion relevancy, a 17.2 percentage point improvement over threshold-based systems, across 12 suggestion categories. The Flutter/Dart dashboard provides real-time visualization at a constant 60 fps on Android and Windows, with FCM push alerts delivered in an average of 1.8 seconds from sensor breach.

Keywords: Internet of Things (IoT), ESP32-WROOM, Environmental Monitoring, Indoor Air Quality (IAQ), MQTT Protocol, ThingSpeak, TensorFlow Lite, Flutter/Dart, Cloud Analytics, Edge Computing, PM2.5, Smart Buildings

CONTENTS

ACKNOWLEDGEMENT			**i**
ABSTRACT			**ii**
LIST OF FIGURES			**vi**
LIST OF TABLES			**vii**
CHAPTER NO		**TITLE**	**PAGE NO**
1		**INTRODUCTION**	**1**
	1.1	**Overview**	**1**
	1.2	**Motivation**	**2**
	1.3	**Problem Identification**	**3**
		1.3.1 Problem statement	**4**
	1.4	**Scope of the Project**	**4**
	1.5	**Objective and Methodology**	**4**
		1.5.1 Objectives	**4**
		1.5.2 Methodology	**5**
	1.6	**Existing System**	**5**
	1.7	**Proposed System**	**6**
	1.8	**Outcome of the Project**	**7**
		1.8.1 Practical applications	**7**
	1.9	**Report Organization**	**8**
	1.10	**Introduction Summary**	**8**
2		**LITERATURE SURVEY**	**9**
	2.1	**Related Work**	**9**
		2.1.1 Reference papers	**9**
		2.1.2 Selection criteria and organization	**9**
		2.1.3 Additional background sources	**9**

2.2	Air Quality Monitoring System Based on IoT Using Raspberry Pi	10
2.3	IoT-Based Air Pollution Monitoring Using ESP32 with MQ-135 and MQ-7	10
2.4	IoT Indoor Air Quality Monitoring System with Android App Integration	11
2.5	Internet of Things Mobile–Air Pollution Monitoring System (IoT- Mob air)	12
2.6	Real-Time Air Pollution Monitoring with Sensors on City Bus	13
2.7	Realizing Mobile Air Quality Monitoring System: Architectural Concept and Device Prototype	13
2.8	Literature Survey Summary	14
3	SYSTEM ANALYSIS	16
3.1	Introduction to System Analysis	16
3.2	Feasibility Study	16
	3.2.1 Technical Feasibility	16
	3.2.2 Economic Feasibility	17
	3.2.3 Operational Feasibility	17
	3.2.4 Schedule Feasibility	18
	3.2.5 Social Feasibility	18
	3.2.6 Risk analysis	18
3.3	System Analysis Summary	19

4	REQUIREMENT ANALYSIS	20
4.1	Functional Requirements	20
4.2	Non-Functional Requirements	21
4.3	Software Requirements	22
	4.3.1 Operating System	22
4.4	Software Requirement Summary	23
4.5	Hardware Requirements	23
4.6	Hardware Requirements Summary	24
4.7	Requirement Analysis Summary	24
5	SYSTEM DESIGN	25
5.1	System Architecture Overview	25
	5.1.1 Layered architecture	25
	5.1.2 System Flow Design	26
	5.1.3 Design Characteristics	27
	5.1.4 Hardware Circuit Design	27
5.2	Gantt Chart	29
	5.2.1 Gantt Chart of Phase One	30
	5.2.2 Gantt Chart of Phase Two	30
5.3	Life Cycle Model	31
5.4	Data Flow Diagram	31
	5.4.1 DFD Level 0	31
	5.4.2 DFD Level 1	32
5.5	Use Case Diagram	33
5.6	Sequence Diagram	34
5.7	Class Diagram	35
5.8	System Design Summary	36

6 SYSTEM IMPLEMENTATION 37
6.1 Modular Approach 37
6.1.1 Sensor Acquisition Module (ESP32 station) 37
6.1.2 ESP32 Processing and Telemetry Module 37
6.1.3 AQI Calculation Module 38
6.1.4 GPS Module 38
6.1.5 Bluetooth Communication Module 39
6.1.6 Telemetry Parsing Module 40
6.1.7 Room Database Module 41
6.1.8 Historical Data Module 42
6.1.9 Android UI Module 42
6.1.10 Graph Visualization Module 44
6.1.11 Gemini AI Module 45
6.1.12 Offline Analysis Module 46
6.2 Programming Code 46
6.2.1 Project structure 47
6.3 System Implementation Summary 47
7 TESTING 48
7.1 Validation and System Testing 48
7.1.1 Software Testing 48
7.1.2 Validation 48
7.1.3 Reasons for Performing Software Validation 48
7.1.4 Test environment and procedure 49
7.1.5 Test design techniques and robustness checks 49
7.2 Test Cases and Testing Summary 50

8 SAMPLE OUTPUT 54

8.1 Snapshots 54

8.1.1 Home / Dashboard 54

8.1.2 Real-Time AQI 55

8.1.3 Environmental Parameters 55

8.1.4 GPS Information 56

8.1.5 Historical Data 56

8.1.6 AQI Trend Graph 57

8.1.7 Gemini AI Analysis 57

8.1.8 Health / Environmental Recommendation 58

8.1.9 Bluetooth Connection 58

8.1.10 Settings / Theme 59

8.2 Hardware Prototype 59

8.3 Sample Output Summary 60

CONCLUSION 61

FUTURE SCOPE 61

REFERENCES 62

LIST OF FIGURES/ILLUSTRATIONS

FIGURE NO.	FIGURE NAME	PAGE NO.
5.1	System Architecture of the Smart Air Quality Monitoring System	24
5.2	System Flow Diagram	25
5.3	Hardware Circuit Diagram of the ESP32 Monitoring Station	26
5.4	Gantt Chart of Phase One (indicative)	29
5.5	Gantt Chart of Phase Two (indicative)	29
5.6	Iterative Incremental Life Cycle Model	30
5.7	DFD Level 0 (Context Diagram)	31
5.8	DFD Level 1	31
5.9	Use Case Diagram	32
5.10	Sequence Diagram	33
5.11	Class Diagram	34
8.1	Dashboard (home) screen	51
8.2	Real-time AQI gauge	52
8.3	Environmental parameter cards and gas-and-dust Summary	53
8.4	GPS information card	53
8.5	History dialog	54
8.6	Gas and dust trend graph	54
8.7	Gemini AI analysis dialog	55

8.8 Health and environmental recommendations 55

8.9 Bluetooth connection banner and device selection 56

8.10 Theme (light and dark) display 56

8.11 Photograph of the assembled hardware prototype 57

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
1.1	AQI categories implemented in the application (AqiCategory)	2
1.2	Comparison of existing and proposed approaches	6
2.1	Air Quality Monitoring System Based on IoT Using Raspberry Pi	10
2.2	IoT-Based Air Pollution Monitoring Using ESP32 with MQ-135 and MQ-7	10
2.3	IoT Indoor Air Quality Monitoring System with Android App Integration	11
2.4	Internet of Things Mobile–Air Pollution Monitoring System(IoT-Mobair)	11
2.5	Real-Time Air Pollution Monitoring with Sensors on City Bus	12
2.6	Realizing Mobile Air Quality Monitoring System: Architectural Concept and Device Prototype	13
2.7	U-Air: When Urban Air Quality Inference Meets Big Data	13
2.8	Applications of Low-Cost Sensing Technologies for Air Quality Monitoring and Exposure Assessment	14
2.9	Summary of reviewed work against project features	15
3.1	Risk analysis and mitigation	18

3.2	Feasibility comparison	18
4.1	Functional requirements	20
4.2	Non-functional requirements	21
4.3	Software requirements	21
4.4	Hardware requirements	23
5.1	Pin connections: circuit diagram versus README	28
5.2	Data dictionary for the Level 1 DFD	32
5.3	Use case descriptions	32
6.1	Source package structure of the Android application	44
7.1	Automated tests present in the repository	46
7.2	Expected AQI category at boundary values (from AqiCategory.fromAqi, with no status text supplied)	47
7.3	Test cases for the Smart Air Quality Monitoring System	48
7.4	Quantities to be measured for validation	49

CHAPTER 1

INTRODUCTION

1.1 Overview

Air pollution is one of the most widespread environmental risks to human health. Particulate matter, combustion by-products, volatile organic compounds and other airborne contaminants are produced by road traffic, industry, construction, cooking and the burning of waste or biomass. Exposure affects the respiratory and cardiovascular systems, and the people most at risk are children, the elderly and those with asthma or heart disease. International guidance such as the World Health Organization air-quality guidelines [4] and national indices such as the Air Quality Index of the US Environmental Protection Agency [9] and the National Air Quality Index of India [13] exist to describe pollution levels in a form that the public can understand.

Conventional air-quality information comes from a limited number of fixed monitoring stations that use reference-grade instruments. These stations are accurate, but they are expensive to install and maintain, and a city usually has only a few of them. Pollution is not uniform: it changes with traffic density, building layout, indoor activity, time of day and weather. A reading from a station several kilometers away therefore says little about the air along a particular road, inside a classroom, or in a kitchen. Low-cost sensing technologies are widely studied as a way to fill this gap, although their limits in accuracy and calibration are well documented [10][11].

The Internet of Things (IoT) makes it possible to build small, inexpensive monitoring nodes that sense, process and transmit data. The ESP32 is a popular choice for such nodes because it combines a dual-core processor, analog-to-digital converters, several hardware serial ports, Wi-Fi and Bluetooth in a low-cost module. When it is combined with a temperature and humidity sensor, a gas sensor, an optical dust sensor and a GPS receiver, it can produce a stream of readings tagged with the location where they were taken.

This project, titled **Smart Air Quality Monitoring System Using IoT and AI** , implements such a station together with a smartphone application. The ESP32 station measures temperature, relative humidity, a gas level from an MQ-135 sensor and a dust density from a Sharp GP2Y1010AU0F sensor, and obtains position data from a NEO-6M GPS module. It sends the data over Bluetooth Classic (Serial Port Profile) to an Android application built

with Kotlin and Jetpack Compose. The application parses the telemetry, shows the air-quality index and the individual parameters, stores every reading in a local Room database, draws historical trend graphs, and uses a Gemini generative-AI model to explain what the readings mean for health and ventilation. When the AI service cannot be reached, a rule-based engine inside the application produces the same kind of structured advice offline.

Table 1.1: AQI categories implemented in the application (AqiCategory)

AQI RANGE	CATEGORY (IN APPLICATION)	HEALTH MESSAGE STORED IN THE APPLICATION
0 - 50	Good	Air quality is satisfactory; little or no risk.
51 - 100	Moderate	Acceptable; very sensitive individuals may experience minor symptoms.
101 - 150	Unhealthy for Sensitive Groups	Sensitive groups may experience health effects.
151 - 200	Unhealthy	Everyone may begin to experience health effects.
201 - 300	Very Unhealthy	Health alert; avoid prolonged outdoor exertion.
301 and above	Hazardous	Emergency conditions; remain indoors.

It is important to be precise about what this index represents. In the application, the index is received from the ESP32 as part of the telemetry packet; the demonstration generator inside the application documents the weighted formula used on the station as AQI = (dust density × 0.6) + (gas value × 0.2) + (temperature × 0.2). This is a composite heuristic indicator derived from low-cost sensors. It is not the pollutant-by-pollutant concentration-to-index conversion used by the official EPA or CPCB indices, although the category bands follow the familiar EPA-style ranges.

1.2 Motivation

The motivation for this work comes from the gap between the importance of air quality and the availability of local, understandable air-quality information. Several observations drive the project:

- **Sparse coverage.** Official stations cannot describe street-level or indoor conditions. A portable station can be carried to the place of interest.
- **Cost.** Reference instruments are beyond the budget of students, small institutions and households. An ESP32 with commodity sensors costs a small fraction of that price.

- **Mobility.** Pollution exposure depends on where a person moves. A device with its own GPS receiver can attach a position to each reading, which is the first step toward mapping and hotspot identification [5][6][8].
- **Interpretation.** Raw numbers such as an ADC count from a gas sensor or a dust density are difficult for non-experts to interpret. A generative-AI assistant can turn the same numbers into plain-language advice, while a rule-based fallback keeps the application useful offline.
- **Learning value.** The project brings together embedded sensing, wireless communication, mobile application development, local databases, data visualization and applied AI in one coherent system, which is appropriate for a final-year engineering project.

1.3 Problem Identification

From the discussion above, the following problems were identified:

1. Stationary monitoring stations provide coarse spatial coverage and cannot capture local variation in pollution or the exposure of a moving person.
2. Commercial portable monitors are costly, often closed, and give little insight into how individual sensor values were obtained or interpreted.
3. Low-cost hobby systems frequently display only raw sensor values on a serial monitor or a cloud dashboard, without a persistent local history and without any explanation aimed at the ordinary user.
4. Systems that depend entirely on Wi-Fi and cloud services stop working when connectivity is poor, which is common outdoors and on the move.
5. Where AI is used, the result is often presented as a prediction without a clear statement of what data it was based on or what it cannot do.

The project therefore asks the following question: can a low-cost ESP32-based station, a smartphone application with local storage, and a generative-AI assistant with an offline fallback be combined into a usable personal air-quality monitoring system, while being honest about the accuracy limits of the sensors?

1.3.1 Problem statement

To design and build a low-cost, portable air-quality monitoring station based on the ESP32 and commodity sensors, and an Android application that receives the station's telemetry over Bluetooth, stores it locally, visualizes it, and interprets it with generative AI and an offline rule-based fallback, while stating clearly the accuracy limits of the sensors used.

1.4 Scope of the Project

The scope of the project is limited to the following implemented functions, which have been verified against the application source code:

- Measurement of temperature, humidity, a gas indicator (MQ-135 analog value), a dust density (GP2Y1010AU0F) and GPS position data on an ESP32-based station.
- Transmission of the telemetry to an Android phone over Bluetooth Classic SPP, with automatic connection to a paired device named ESP32_AQI or a device selected from a paired-device dialog.
- Display of the air-quality index, the four environmental parameters and the GPS information (latitude, longitude, altitude, speed, number of satellites) in the application.
- Local storage of every received reading in a Room database, a history viewer, and a gas-and-dust trend graph with selectable number of points.

The following are outside the present scope: calibration of the sensors against reference instruments, regulatory-grade measurement, forecasting or prediction of future air quality, cloud storage and multi-user dashboards, and map-based pollution mapping across many devices. Some of these are proposed as future work in the Future Scope section.

1.5 Objective and Methodology

1.5.1 Objectives

1. To design and assemble an ESP32-based environmental monitoring station using temperature/humidity, gas, dust and GPS sensing.
2. To transmit the measured telemetry wirelessly to a smartphone using Bluetooth Classic SPP.
3. To develop an Android application using Kotlin and Jetpack Compose that parses, displays and stores the telemetry.

4. To provide historical monitoring through a Room database, a history viewer and trend graphs.
5. To provide AI-based environmental analysis and recommendations using Gemini, with an offline rule-based fallback.

1.5.2 Methodology

An iterative and incremental approach was followed. The requirements were first derived from the problem statement, the hardware components were selected, and the architecture was designed as a pipeline from sensing to presentation. The hardware was then assembled on a breadboard, the telemetry link was established, and the Android application was built incrementally: Bluetooth connection first, then telemetry parsing and the dashboard, then Room persistence, then charts and the GPS card, and finally the AI analysis and the offline engine. Each increment was checked on its own before the next was added. The application follows the MVVM architecture with the repository pattern so that the user interface, the data layer and the device communication remain separate.

1.6 Existing System

Existing air-quality monitoring can be grouped into three kinds of systems. The first is the network of fixed government monitoring stations, which use reference instruments, give reliable regional data, and publish an index through websites and applications. The second is commercial portable or consumer air-quality monitors, which are convenient but can be expensive, are often tied to a vendor application, and do not expose the processing to the user. The third is the large body of IoT research prototypes that connect gas and dust sensors to Arduino, Raspberry Pi or ESP32 boards and push the data to cloud platforms such as ThingSpeak or Firebase [1][3][12]. Some of these prototypes also provide an Android application or a GPS-based route view [1][8], and a number of studies place sensors on public-transport vehicles to obtain mobile coverage [5][6].

The limitations of these existing arrangements, as relevant to this project, are summarized below.

- Fixed stations are sparse and give no information about a person's immediate surroundings.
- Many prototypes rely on Wi-Fi and a cloud service and therefore provide no data when the connection is lost.

- Raw values are shown with little interpretation, so a non-specialist cannot decide what action to take.
- Several low-cost systems do not store a local history on the phone, so trends cannot be examined offline.

1.7 Proposed System

The proposed system is a portable monitoring station and a companion Android application connected by Bluetooth Classic. The conceptual pipeline, verified against the application source code, is environmental sensors → ESP32 → sensor data acquisition → data processing and AQI calculation → GPS data → Bluetooth Classic SPP → Android application → telemetry parsing → Room database → dashboard, charts and history → Gemini AI analysis → environmental and health insights → user.

The station samples temperature and humidity (DHT sensor), a gas indicator (MQ-135) and dust density (GP2Y1010AU0F), reads latitude, longitude, altitude, speed and satellite count from the NEO-6M GPS, calculates the composite index and category, and sends the values to the phone. In the application, a Bluetooth manager reads the incoming stream, a parser converts each valid line into an AirQualityData object, and a View Model publishes the state to a Jetpack Compose dashboard while a repository saves each reading to the Room database. The dashboard contains the AQI gauge, the four parameter cards, a gas-and-dust AI summary card, the historical trend chart, and the GPS card. An AI Analysis button produces the overall assessment. Table 1.2 compares the proposed system with the existing arrangements described above.

Table 1.2: Comparison of existing and proposed approaches

ASPECT	FIXED STATIONS	TYPICAL CLOUD-BASED IOT PROTOTYPE	PROPOSED SYSTEM
Coverage	Fixed, sparse locations	Fixed or mobile node	Portable node with its own GPS
Connectivity	Network infrastructure	Wi-Fi and cloud required	Bluetooth Classic to phone; cloud only for AI
Local history	Not applicable to the user	Often cloud only	Room database on the phone
Interpretation	Index and category	Raw values or charts	Gauge, charts, AI analysis, offline rules
Offline operation	Not applicable	Usually not possible	Dashboard, history and rule-based advice work offline

ASPECT	FIXED STATIONS	TYPICAL CLOUD-BASED IOT PROTOTYPE	PROPOSED SYSTEM
Measurement quality	Reference grade	Low-cost, uncalibrated	Low-cost, uncalibrated (limitation stated)

1.8 Outcome of the Project

The expected and demonstrable outcomes of the project are as follows.

- A working hardware station design (as shown in the circuit diagram) that combines the four sensing functions with an ESP32 and a battery-powered supply.
- An Android application, identified in the manifest as the package com.aistudio.airquality.aesp (application name “Smart AQI”), that supports Android 7.0 (API 24) and above and targets API 36.
- Real-time display of the AQI, temperature, humidity, gas value, dust density, and GPS information with six color-coded AQI categories.
- Persistent local storage and a history viewer, and a gas-and-dust trend graph with 15-, 30-or 50-point views and an option to hide either series.

1.8.1 Practical applications

The system is intended for indicative monitoring in the following situations, always with the understanding that the readings are not regulatory measurements:

- **Classrooms and laboratories** , where ventilation and dust build-up can be observed over a lesson or a day.
- **Homes and kitchens** , to see the effect of cooking, cleaning or burning incense on gas and dust readings.
- **Workshops and small industries** , to compare working areas and notice dust or fumes.
- **Education and demonstration** , as a teaching platform that combines embedded systems, mobile development, databases and AI.

A short-range Bluetooth link was chosen over Wi-Fi for the core data path because it needs no router or data plan, pairs once with a phone and keeps the personal readings on the owner's device. The trade-off is a range of the order of a few metres to a few tens of metres, a single connected phone, and no remote viewing; these are acceptable for a personal monitor and are discussed again in the conclusion.

1.9 Report Organization

This report is organized into eight chapters followed by the conclusion, future scope and references.

- **Chapter 1 - Introduction** presents the background, motivation, problem, scope, objectives, existing and proposed systems and outcomes.
- **Chapter 2 - Literature Survey** reviews eight verified studies on IoT air-quality monitoring, mobile sensing, low-cost sensor quality and AI-based analysis.
- **Chapter 3 - System Analysis** discusses technical, economic, operational, schedule and social feasibility.
- **Chapter 4 - Requirement Analysis** lists functional, non-functional, software and hardware requirements.
- **Chapter 5 - System Design** describes the architecture, hardware circuit, flow, Gantt charts, life-cycle model, data flow diagrams and UML diagrams.
- **Chapter 6 - System Implementation** explains the modules and presents selected code from the repository.
- **Chapter 7 - Testing** defines the validation approach and the test cases, with results that remain to be recorded.
- **Chapter 8 - Sample Output** describes the application screens and provides places for actual screenshots.
- **Conclusion, Future Scope and References** summarize the work, its limits and the way forward.

1.10 Introduction Summary

This chapter introduced the problem of limited and hard-to-interpret air-quality information and explained why a portable, low-cost, smartphone-connected monitoring station is useful. The proposed system uses an ESP32 with DHT, MQ-135, GP2Y1010AU0F and NEO-6M sensing, Bluetooth Classic SPP to an Android application, Room storage for history, charts for trends and a Gemini-based analysis with an offline fallback. The chapter also stated clearly that the index is a heuristic derived from low-cost sensors and that the AI component analyses current readings rather than predicting future air quality. The next chapter surveys the published work that relates to these ideas.

CHAPTER 2

LITERATURE SURVEY

2.1 Related Work

2.1.1 Reference papers

The literature relevant to this project spans several connected areas: IoT-based air-quality monitoring with low-cost microcontrollers, ESP32-based gas and dust sensing, mobile and vehicle-mounted pollution monitoring, GPS-tagged measurements, the quality and limits of low-cost sensors, and data-driven or AI-based analysis of air quality. Eight studies were selected because each of them informs a specific design decision in this project. Their bibliographic details were checked against publisher, repository or indexing pages, and every statement made about them below is based on the abstract or the published description of the work. No numerical results from these papers are reproduced, because the aim of the survey is to identify design approaches and gaps rather than to compare accuracy figures obtained under different conditions.

2.1.2 Selection criteria and organization

The papers are presented in four groups. The first group (Sections 2.2 to 2.4) covers IoT monitoring systems built from low-cost sensors, including ESP32-based systems and one with an Android application. The second group (Sections 2.5 to 2.6) covers mobile and GPS-aware monitoring. The third (Section 2.7) represents data-driven inference of air quality. The fourth (Section 2.8) addresses the performance and limitations of low-cost sensors, which directly determine how the results of this project must be interpreted. Each study is summarized in a table in the format NAME, YEAR, AUTHOR, FEATURE, ADVANTAGE and DISADVANTAGE, followed by a short discussion of its relevance. The advantage and disadvantage entries are the authors' assessment relative to the objectives of the present project and are not claims made by the original authors.

2.1.3 Additional background sources

A broader survey by Mokrani et al. [7] reviews IoT-based air-quality monitoring, including the pollutants of concern, the sensing equipment and possible IoT configurations, and it was used to confirm that the sensor-microcontroller-wireless-application pattern adopted here is a common and accepted architecture. Official documents such as the EPA technical

assistance document on the AQI [9], the Indian National Air Quality Index [13] and the WHO global air-quality guidelines [4] were used for the category definitions and health context.

2.2 Air Quality Monitoring System Based on IoT Using Raspberry Pi

Table 2.1: Air Quality Monitoring System Based on IoT Using Raspberry Pi

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
IoT air-quality monitoring with Raspberry Pi	2017	S. Kumar, A. Jasuja [12]	Raspberry Pi connected to gas and particulate sensors; measures pollutants such as PM, CO and CO2; data is sent to a cloud platform for real-time monitoring and analysis.	Low-cost single-board computer; remote real-time access to readings through the cloud; shows that commodity sensors can be combined into an IoT monitoring node.	Depends on network connectivity and a cloud service for data access; a single-board Linux computer is heavier in power and cost than a microcontroller; no phone-side local history is described.

This early and frequently cited study demonstrates the basic IoT pattern for air-quality monitoring: sensors feed a small computer, and the data is made available remotely through a cloud service. It established that affordable hardware can monitor several pollutants in real time. For the present project, the main lesson is architectural. The sensing node does the data collection, and a separate application presents the data. However, the reliance on a Raspberry Pi and a cloud connection leads to higher power consumption and a network dependency, which are less suitable for a battery-powered portable node.

2.3 IoT-Based Air Pollution Monitoring Using ESP32 with MQ-135 and MQ-7

This study is close to the present project in its choice of the ESP32 and the MQ-135 sensor. It reports that a portable device measuring CO and CO2 equivalents can be built cheaply and used at multiple spots in a campus, with the data uploaded to a cloud platform and examined later through a web application. It supports the technical feasibility of the ESP32-plus-MQ-135 combination used in this project.

Table 2.2: IoT-Based Air Pollution Monitoring Using ESP32 with MQ-135 and MQ-7

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
IoT-based Air Pollution Monitoring System ... using MQ135 and MQ7 Sensors	2022	Y. A. U. Dilusha, T. Priyanka, M. Zahir, T. Yogamalar [3]	Low-cost portable ESP32 device with MQ-135 and MQ-7 sensors for CO2 and CO; measured over five days at different spots in an education campus; data sent to ThingSpeak and shown on a web application.	Shows ESP32 plus MQ-series sensors is a practical low-cost, portable design; field data was collected at several locations.	Gas sensing only (abstract does not mention dust or GPS); depends on ThingSpeak and the internet; MQ-series sensors need calibration for quantitative use.

Two gaps are visible. The system is limited to gas sensing and does not include particulate sensing, and all viewing is through cloud services. The proposed system adds an optical dust sensor, temperature and humidity sensing and GPS, and it keeps a local database on the phone. It also treats the MQ-135 output as a general gas and air-quality indicator and not as a calibrated measurement of a specific gas, which is consistent with the datasheet description of the sensor as a sensitive device for a mixture of gases.

2.4 IoT Indoor Air Quality Monitoring System with Android App Integration

This recent work is the closest published parallel to the present project because it joins an ESP32 node, several sensor types and an Android application. It shows that smartphone presentation is a natural way to deliver air-quality data to users, and that actuation based on thresholds can be added. It was tested in laboratories and classrooms, which are indoor environments.

In that design the application obtains data through a cloud platform. The present project differs in three ways: the phone connects directly to the node over Bluetooth Classic, so readings are available without an internet service; GPS data accompanies each reading so that mobile use is possible; and a generative-AI assistant with a rule-based fallback explains the readings. The present project does not include actuation such as controlling a fan.

Table 2.3: IoT Indoor Air Quality Monitoring System with Android App Integration

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
IoT Indoor Air Quality Monitoring System with Android App Integration	2025	S. G. Krishnan, W. N. H. Wan Kairuddin [1]	ESP32 with MQ-135, MQ-7, PM2.5 and DHT22 sensors; data to ThingSpeak and displayed on a custom Android app; an extractor fan switches on above a threshold; tested in laboratories and classrooms.	Combines gas, particulate, temperature and humidity sensing with an Android app; includes an automatic actuation example.	Indoor-focused; depends on ThingSpeak and internet access for the app; abstract does not mention GPS or local on-phone storage or AI interpretation.

2.5 Internet of Things Mobile–Air Pollution Monitoring System (IoT-Mobair)

Table 2.4: Internet of Things Mobile–Air Pollution Monitoring System (IoT-Mobair)

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
IoT-Mobair	2019	S. Dhingra, R. B. Madda, A. H. Gandomi, R. Patan, M. Daneshmand [8]	Gas sensors with a microcontroller and Wi-Fi module send data to the cloud; an Android application shows air-quality data; pollution along a travel route is predicted and a warning displayed; AQI data can be used to predict future levels.	Combines AQI, GPS and an Android application; introduces pollution-aware route information similar to a navigation service.	Needs Wi-Fi and cloud; relies on gas sensors of limited precision; prediction component requires accumulated data from many locations.

IoT-Mobair is important because it links a sensing kit, the AQI and an Android application with location information. It shows that users value air-quality information tied to where they are and where they are going. The authors also describe route-level pollution estimation and future AQI prediction using accumulated cloud data.

The present project is a smaller, device-centred version of this idea. Each reading is stored with its GPS coordinates, which is the data a route or map feature would need, but the project

does not implement route prediction or AQI forecasting. For this reason the Gemini component is described throughout this report as environmental analysis and recommendation based on current readings, and not as prediction.

2.6 Real-Time Air Pollution Monitoring with Sensors on City Bus

Table 2.5: Real-Time Air Pollution Monitoring with Sensors on City Bus

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Real-time air pollution monitoring with sensors on city bus	2020	S. Kaivonen, E. C.-H. Ngai [6]	Low-cost wireless sensors on public-transport buses in Uppsala, integrated with the GreenIoT testbed; complements stationary sensors and the single ground-level monitoring station; evaluates communication quality and data quality.	Fine-grained, real-time coverage using existing vehicles; shows mobile sensing complements fixed stations.	Requires vehicle installation and network infrastructure; moving low-cost sensors raise data-quality questions; coverage limited to bus routes.

Kaivonen and Ngai report an experimental study in which low-cost sensors on city buses supplement fixed sensors and the only reference-grade station in the city. It supports the central motivation of this project: fixed stations are too sparse to describe local air quality, and mobile sensors can add useful detail. The study also examines communication and data quality, which reminds the designer that mobile low-cost data must be handled carefully.

2.7 Realizing Mobile Air Quality Monitoring System: Architectural Concept and Device Prototype

This paper proposes a layered architecture for mobile air-quality monitoring with sensing, communication and application layers, and describes a device that knows its own position through GPS and sends data through Wi-Fi and 4G. Its layered view is directly reflected in the architecture of the present project, where the ESP32 station is the sensing layer, Bluetooth Classic is the communication layer and the Android application is the application layer.

Table 2.6: Realizing Mobile Air Quality Monitoring System: Architectural Concept and Device Prototype

NAME	YEAR	AUTHOR	FEATURE	ADVANTAGE	DISADVANTAGE
Realizing Mobile Air Quality Monitoring System	2021	V. A. Nguyen, V. H. Vu, V. S. Doan, T. H. Nguyen, D. P. Thuan, K. Nguyen, P. L. Nguyen, M. T. Le [5]	Three-tier architecture (sensing, communication, application); device installed on vehicles; location from GPS; data sent over Wi-Fi and 4G (LTE); trial deployment on a vehicle in Hanoi.	Overcomes the limits of stationary monitoring; GPS-tagged measurements; clear layered architecture.	More complex hardware and power design; needs Wi-Fi or cellular connectivity; vehicle-mounted rather than personal.

2.8 Literature Survey Summary

Table 2.9 summarizes how each study relates to the elements of the proposed system and identifies the gap that the present project addresses.

Table 2.9: Summary of reviewed work against project features (as stated in each source abstract; '-' means not stated)

STUDY	ESP32 / MCU	DUST + GAS + T/H	GPS	PHONE APP	LOCAL PHONE STORAGE	AI INTERPRETATION
[12] Kumar & Jasuja	Raspberry Pi	Gas, PM	-	-	-	-
[3] Dilusha et al.	ESP32	Gas	-	Web app	-	-
[1] Krishnan et al.	ESP32	Gas, PM, T/H	-	Android	-	-
[8] Dhingra et al.	Microcontrol ler	Gas	Yes	Android	-	Prediction (cloud)
[6] Kaivonen & Ngai	Mobile node	Low-cost sensors	Mobile	-	-	-
[5] Nguyen et al.	Vehicle device	Ambient sensors	Yes	-	-	-
Proposed	ESP32	Gas, dust, T/H	Yes	Android	Room DB	Gemini + offline rules

The survey shows that the sensor-microcontroller-application architecture is well established and that mobile and GPS-aware monitoring is of recognized value. Most of the ESP32-based studies reviewed rely on cloud platforms for viewing the data, and none of the abstracts

examined describes a combination of direct Bluetooth delivery to the phone, a local on-device history, GPS tagging and generative-AI interpretation with an offline fallback. The literature also repeatedly warns that low-cost sensors need characterisation. The proposed system is positioned in this gap, while remaining honest about its sensor limitations.

CHAPTER 3

SYSTEM ANALYSIS

3.1 Introduction to System Analysis

System analysis is the study of a proposed system to decide what it must do, whether it can be built with the resources available, and how it will fit its users and environment. For this project the analysis considers two cooperating subsystems: the ESP32 sensing station and the Android application. The station contributes the hardware components (ESP32, DHT sensor, MQ-135, GP2Y1010AU0F, NEO-6M, DC-DC converter, 18650 cell) and the application contributes the software layers (Bluetooth manager, parser, Room database, ViewModel, Compose user interface and AI analysis). The feasibility study that follows examines the five usual dimensions for each of these elements. The result is summarized in Table 3.2 at the end of this chapter.

3.2 Feasibility Study

A feasibility study determines whether the project is worth attempting and can be completed with the available time, money, skills and technology. The following subsections apply it to the ESP32, the sensors, GPS, Bluetooth, Android, Room and Gemini AI.

3.2.1 Technical Feasibility

- **ESP32.** The ESP32 provides analog-to-digital converters, multiple UARTs, GPIO pins and a Bluetooth Classic controller in one module, so a single board can read the analog sensors, communicate with the GPS receiver and stream data to a phone. The circuit diagram shows the board used in a common development-board form with labelled pins.
- **Sensors.** The DHT sensor gives digital temperature and humidity. The MQ- 135 gives an analog output that changes with the concentration of a mixture of gases. The GP2Y1010AU0F is an optical dust sensor with an analog output and an LED that must be pulsed. Each of these is available as a low-cost module and is well supported by libraries and documentation. Technically, all three can be read on an ESP32, although the analog sensors need attention to voltage levels (Section 5.1.4).
- **GPS.** The NEO-6M is a u-blox GPS receiver that streams NMEA sentences over a serial interface. The circuit uses the ESP32 second hardware serial port (RX2 and TX2 labels).

- **Bluetooth.** Bluetooth Classic with the Serial Port Profile is supported by Android through RFCOMM sockets, and the application uses the standard SPP UUID. This is a mature and well-documented technique.
- **Android and Compose.** Kotlin, Jetpack Compose and the MVVM pattern are the current recommended approach for Android development.
- **Room.** Room is the standard Android abstraction over SQLite and supports Flow-based reactive queries.
- **Gemini AI.** The Gemini API and the Firebase AI Logic SDK allow an Android application to send a prompt and receive generated text from the Gemini model family [2]. Because network access and API availability cannot be guaranteed, the project adds a rule-based fallback.

The project is therefore technically feasible. The main technical risks are sensor calibration and the voltage compatibility of the analog sensor outputs with the ESP32 inputs, both of which are discussed later in the report.

3.2.2 Economic Feasibility

The hardware uses commodity components: an ESP32 development board, a DHT sensor, an MQ-135 module, a GP2Y1010AU0F sensor, a NEO-6M module with antenna, a DC- DC converter module, an 18650 cell, a capacitor, a resistor, a breadboard and wires. These are widely available at low prices compared with reference-grade instruments. The exact cost depends on the supplier and date and has not been recorded in the project documentation, so the bill of materials in Chapter 4 leaves the price column as [Insert actual cost]. The software uses free and open tools: Android Studio, Kotlin, Jetpack Compose, Room and Firebase tooling. The Gemini API is an optional paid or quota-limited service; the application continues to work without it through the offline engine. The project is therefore economically feasible within an academic budget.

3.2.3 Operational Feasibility

The operation of the system is simple for the user. The station is switched on and paired once in the Android Bluetooth settings. In the application, one tap on the Bluetooth icon connects to a paired device named ESP32_AQI, and the dashboard begins to update. The AI Analysis button produces an explanation in a dialog. The history dialog and the trend chart require no configuration. A demo mode lets the user try the application with no hardware. The offline behavior means that the system remains useful at locations without internet access.

3.2.4 Schedule Feasibility

The work was divided into two phases (Section 5.2). Phase One covers topic selection, the literature survey, requirement analysis, component selection and system design. Phase Two covers circuit assembly, the telemetry link, the Android application, charts, the AI module, testing and the final report. The modular structure allowed the hardware and software to progress in parallel.

3.2.5 Social Feasibility

Air quality is a matter of public concern, and a low-cost personal monitor has clear social value in raising awareness of local pollution and the need for ventilation. The application presents health messages in cautious, general terms aligned with common AQI categories and does not provide medical diagnosis. Because the readings come from uncalibrated sensors, the application and this report make clear that the values are indicative. No personal data other than the sensor readings and the station's coordinates is handled, and these are stored on the phone.

3.2.6 Risk analysis

Feasibility also depends on whether the principal risks can be managed. Table 3.1 lists the risks identified during the analysis and the mitigation adopted or recommended.

Table 3.1: Risk analysis and mitigation

RISK	IMPACT	MITIGATION
Low-cost sensor inaccuracy and drift	Index is only indicative	State the limitation; warm-up; calibrate against a reference (future work)
5 V sensor output on a 3.3 V ESP32 input	Possible damage or clipped readings	Measure outputs; add a voltage divider if needed
Missing common ground or unclear ESP32 power in the circuit	Erratic analog readings	Verify on the physical prototype
Bluetooth range and dropouts	Gaps in the record	Automatic reconnection by pressing connect; keep phone near the station
No GPS fix indoors	Zero coordinates in readings	Fix check in the application; map button disabled with a message
AI service unavailable, quota exhausted or key missing	No AI text	Rule-based fallback in both AI classes
API key exposure	Misuse of the key	Key held in a local .env file and BuildConfig; not committed (example file only)
Chart scripts not loadable offline	Chart does not render	SVG fallback in the page and a native Compose chart

Table 3.2 summarizes the feasibility assessment for each element.

Table 3.2: Feasibility comparison

ELEMENT	TECHNICAL	ECONOMIC	OPERATIONAL	REMARKS
ESP32	Feasible: ADC, UART, Bluetooth Classic	Low cost	Simple to power and flash	Core controller
Sensors (DHT, MQ-135, GP2Y1010)	Feasible; analog levels need care	Low cost	Warm-up and placement matter	Indicative, uncalibrated
GPS (NEO-6M)	Feasible via UART/NMEA	Low cost	Needs open sky for a fix	Position, altitude, speed, satellites
Bluetooth SPP	Feasible; standard UUID	No cost	One-time pairing	Single phone, short range
Android + Compose	Feasible; MVVM	Free tools	Familiar to users	minSdk 24, targetSdk 36
Room database	Feasible; Flow queries	Free	Automatic saving	Offline history
Gemini AI	Feasible; needs network/key	Quota or paid use	Fallback keeps app usable	Analysis, not prediction

3.3 System Analysis Summary

The analysis shows that every component of the proposed system is available, affordable and well supported, and that the system can be built and operated within the resources of an academic project. The principal risks are the accuracy of low-cost sensors, which is handled by describing the results as indicative, and the dependence of the AI feature on network access, which is handled by an offline rule-based engine. The next chapter converts these conclusions into specific functional, non-functional, software and hardware requirements.

CHAPTER 4

REQUIREMENT ANALYSIS

Requirement analysis converts the feasibility conclusions of Chapter 3 into a precise list of what the system must do and what it needs in order to run. Only features that are present in the application source code are listed as functional requirements. The hardware requirements are taken from the circuit diagram supplied with the project.

4.1 Functional Requirements

Table 4.1 lists the functional requirements. The final column names the source element in which the function is implemented, so that each requirement can be traced to the repository.

Table 4.1: Functional requirements

ID	FUNCTIONAL REQUIREMENT	IMPLEMENTED IN (SOURCE ELEMENT)
FR-1	Connect to the ESP32 over Bluetooth Classic SPP, either automatically to the paired device named ESP32_AQI (or a device whose name contains ESP32 or AQI) or by choosing from the paired-device dialog	Esp32BluetoothManager, ConnectionBanner, DeviceSelectionDialog
FR-2	Request the required Bluetooth permissions at run time before connecting	ConnectionBanner (permission launcher)
FR-3	Receive newline-delimited JSON telemetry and ignore lines that are not complete JSON objects	Esp32BluetoothManager
FR-4	Parse telemetry into temperature, humidity, gas, dust, AQI, category, latitude, longitude, altitude, speed and satellites	AirQualityData.fromJson
FR-5	Display the AQI on a gauge with a colour-coded category	AqiGaugeCard, AqiCategory
FR-6	Display temperature and humidity	SensorMetricsGrid
FR-7	Display the MQ-135 gas value (ADC units) and the dust density (µg/m³)	SensorMetricsGrid
FR-8	Display GPS information (latitude, longitude, altitude, speed, satellites) and open the location in an external map application	GpsMapCard
FR-9	Store every received reading in a local database	AirQualityRepository, AirQualityDao
FR-10	Show the last stored reading on start-up, and a history list with timestamps; allow the history to be cleared	HistoryDialog, ViewModel.clearHistory
FR-11	Plot gas and dust history as a trend graph with series toggles and a 15, 30 or 50 point view, using Recharts or a native Compose canvas	GasDustTrendChartCard

ID	FUNCTIONAL REQUIREMENT	IMPLEMENTED IN (SOURCE ELEMENT)
FR-12	Request an overall AI analysis (summary, key concerns, health impact, recommendations, safety level)	GeminiAirQualityAdvisor, AiAnalysisDialog
FR-13	Request a gas-and-dust AI summary (gas status, dust status, verdict, recommendation)	FirebaseGasDustAnalyzer, GasDustSummaryCard
FR-14	Provide offline analysis when the AI service, network or API key is unavailable	generateLocalExpertAnalysis, generateOfflineAnalysis

Note: the theme choice is held in the ViewModel state and the source contains no code that saves it between launches, so it is not listed as a persistent setting.

4.2 Non-Functional Requirements

The non-functional requirements describe the qualities that the system should have. Where a quantitative target would normally be stated, none is given, because no measurements have been made for this report; such targets are marked [To be measured].

Table 4.2: Non-functional requirements

QUALITY	REQUIREMENT	HOW ADDRESSED IN THE CURRENT IMPLEMENTATION
Performance	Telemetry should reach the screen without visible lag; the user interface must stay responsive.	Bluetooth reading runs on the IO dispatcher; state is published through StateFlow; the demo stream is generated every 5 seconds.
Reliability	The application must survive malformed data, disconnections and AI failures.	Non-JSON lines are ignored; parse errors return null; connection errors are mapped to an Error state; AI failures fall back to local analysis.
Maintainability	Code should be modular and testable.	MVVM with repository; separate data, ui and theme packages; Compose components with test tags.
Scalability	The data model should be extendable to new sensors.	AirQualityData and the Room entity can gain fields; Room database is versioned (version 1).
Security	API keys must not be hard-coded in source; sensitive data should be limited.	Key is read from BuildConfig through the Secrets Gradle plugin and .env file; readings are stored locally.
Portability	Run on a broad range of Android phones.	minSdk 24 (Android 7.0), targetSdk 36.
Offline capability	Core monitoring must work without the internet.	Bluetooth link, Room storage, dashboard, native chart and rule-based analysis work offline.

Two limitations related to these requirements should be noted. First, the Recharts view loads React and Recharts scripts from an external content-delivery network; when these scripts cannot be loaded, the embedded page draws a simpler SVG chart, and the native Compose chart can be selected explicitly

4.3 Software Requirements

The software requirements are derived from the Gradle build files and the README of the repository. Table 4.3 lists the tools and libraries and the versions declared in the project.

Table 4.3: Software requirements

CATEGORY	SOFTWARE / LIBRARY	VERSION OR DETAIL (AS DECLARED)
Development environment	Android Studio	Meerkat or newer (README)
Language	Kotlin; Kotlin Coroutines	Kotlin 2.2.10; coroutines 1.10.2
Build	Android Gradle Plugin; KSP; Secrets Gradle Plugin; Google Services plugin	AGP 9.1.1; Java 11 source/target compatibility; JDK 17 or 21 for building (README)
UI	Jetpack Compose, Material 3, Material Icons Extended	Compose BOM 2024.09.00; Activity Compose 1.10.1
Architecture	AndroidX Lifecycle ViewModel and runtime	2.8.7
Database	Room (runtime, ktx, compiler)	2.7.0 (SQLite)
Networking	OkHttp; Retrofit and Moshi (declared)	OkHttp 4.10.0; Retrofit 2.12.0; Moshi 1.15.2
AI	Firebase BOM and firebase-ai; Gemini via REST	Firebase BOM 34.17.0
Charts	Recharts 2.12.7 with React 18.2.0 in a WebView; Compose Canvas	Loaded from a CDN at run time; native fallback
Testing	JUnit 4, Robolectric, Compose UI test, Roborazzi	JUnit 4.13.2; Robolectric 4.16.1; Roborazzi 1.59.0
Station firmware	Arduino-style C++ using BluetoothSerial (per README)	Firmware source is not part of the repository

4.3.1 Operating System

The application runs on Android 7.0 (API level 24) or later, as declared by minSdk 24, and is built against API 36. On Android 12 and later the user must allow the Nearby devices permissions (Bluetooth connect and scan) when asked; on earlier versions the application requests location permission, which the platform historically required for Bluetooth device discovery. Development was carried out on a desktop or laptop operating system that

supports Android Studio (Windows, macOS or Linux); the specific operating system used by the team is not recorded in the project documentation: [Insert development OS].

4.4 Software Requirement Summary

- The mobile application is implemented entirely in Kotlin using Jetpack Compose for the user interface and the MVVM pattern for state management.
- Local persistence uses Room over SQLite, and the database is named smart_air_quality.db with a single table, air_quality_readings.
- Device communication uses the Android Bluetooth Classic API with the SPP UUID 00001101-0000-1000-8000-00805F9B34FB.
- AI analysis uses two routes: a REST call to a Gemini model endpoint in GeminiAirQualityAdvisor and the Firebase AI SDK in FirebaseGasDustAnalyzer. Both fall back to rule-based analysis.
- The Gemini key is supplied through a .env file (GEMINI_API_KEY) processed by the Secrets Gradle plugin; the example file contains a placeholder value.

4.5 Hardware Requirements

The hardware components were identified from the circuit diagram (Figure 5.3). Table 4.4 lists them. Prices and exact part numbers that cannot be read from the diagram are left as placeholders.

Table 4.4: Hardware requirements

NO.	COMPONENT	ROLE IN THE SYSTEM	DETAIL VERIFIED FROM THE CIRCUIT DIAGRAM
1	ESP32 development board (ESP-WROOM-32 module)	Main controller; reads sensors, computes index, streams data over Bluetooth	Pin labels visible: 3V3, GND, D15, D2, D4, RX2, TX2, D5, D18, D19, D21, RX0, TX0, D22, D23, and Vin, GND, D13 ... D34, VN, VP, EN
2	DHT temperature and humidity sensor module	Temperature and humidity	Blue-case module with + / out / - pins; the exact model (DHT11 or DHT22) is not stated in the diagram
3	MQ-135 gas sensor module	Gas / air-quality indicator (analog)	Four terminals including VCC, GND and analog output
4	Sharp GP2Y1010AU0F optical dust sensor	Dust density (analog output and pulsed LED)	Six-pin connector; 220 µF capacitor and a resistor labelled 20.0 Ω shown with the LED circuit
5	NEO-6M GPS module with antenna	Location, altitude, speed, satellites	Pins VCC, RX, TX, GND

NO.	COMPONENT	ROLE IN THE SYSTEM	DETAIL VERIFIED FROM THE CIRCUIT DIAGRAM
6	DC-DC converter module	Power conversion from the battery	Terminals VIN+, VIN-, VOUT+, VOUT-; output voltage setting is not visible
7	18650 Li-ion cell	Battery power source	Shown with - and + terminals
8	Capacitor 220 µF	Supply smoothing for the dust-sensor LED circuit	Labelled 220 µF
9	Resistor	Series resistor in the dust-sensor LED line	Labelled 20.0 Ω in the diagram
10	Breadboard and jumper wires	Interconnection and power rails	Two power rails used for the converter output

In addition, a smartphone running Android 7.0 or later with Bluetooth is required. A phone with internet access is needed only for cloud-based AI analysis. Cost of each component: [Insert actual cost].

4.6 Hardware Requirements Summary

The station is a compact breadboard prototype. The ESP32 is the only programmable element and has to provide two analog inputs, one pulsed output, one digital sensor input and one hardware serial port. The two analog sensors and the GPS receiver draw their power from the converter output rails, while the DHT module is powered from the 3V3 and GND pins of the ESP32 according to the diagram. The way in which the ESP32 itself is powered is not shown: the Vin pin is not wired in the diagram, so power is presumably supplied through its USB connector. This point is flagged for confirmation in Section 5.1.4.

4.7 Requirement Analysis Summary

This chapter defined sixteen functional requirements that are traceable to the repository, eight groups of non-functional requirements, the software tools and libraries with their declared versions, and the hardware components identified from the circuit diagram. Items missing from the available material (prices, DHT model, converter voltage, firmware source, development OS) are left as marked placeholders.

CHAPTER 5

SYSTEM DESIGN

5.1 System Architecture Overview

5.1.1 Layered architecture

The system is organized as three layers connected by a short-range wireless link. The **sensing layer** is the ESP32 station with its four sensing functions. The **communication layer** is the Bluetooth Classic Serial Port Profile link between the ESP32 and the phone. The **application layer** is the Android application, which contains the device manager, the telemetry parser, the local database, the user interface and the AI analysis modules. Figure 5.1 shows the architecture. The layering follows the sensing, communication and application organization used in the mobile monitoring literature [5].

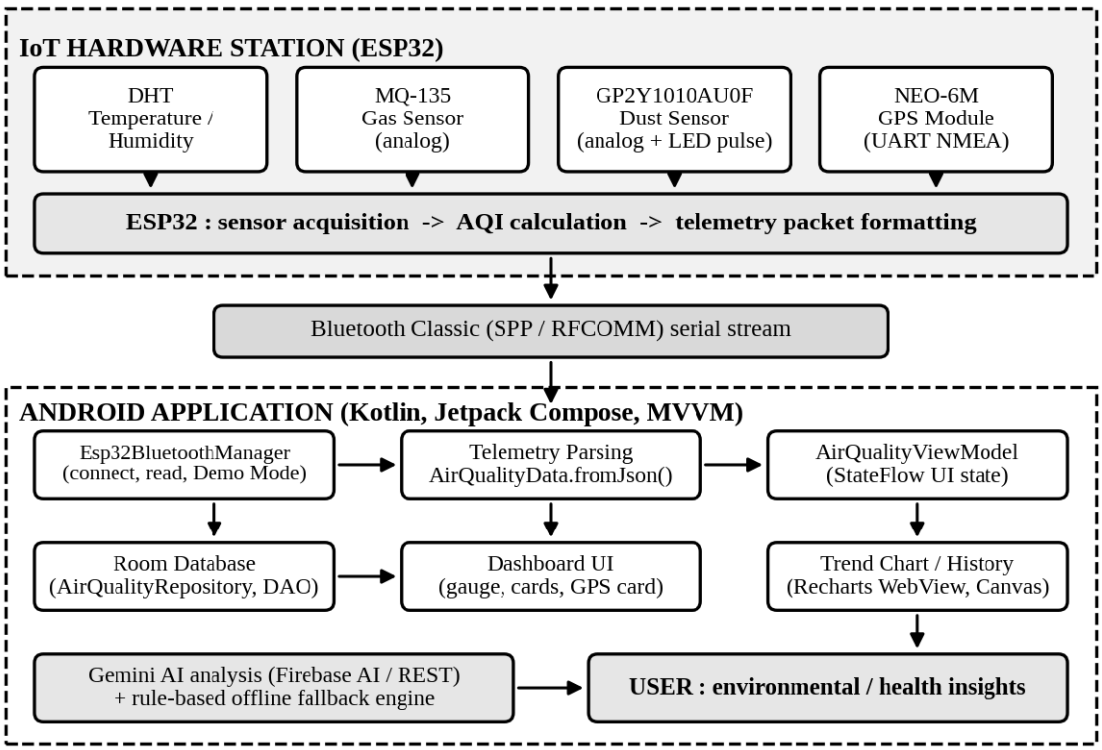


Figure 5.1: System Architecture of the Smart Air Quality Monitoring System

The data path, verified against the source code, is as follows. The environmental sensors and the GPS module feed the ESP32. The ESP32 acquires the readings, calculates the AQI and sends one JSON text line per sample. On the phone, Esp32BluetoothManager reads the stream line by line, AirQualityData.fromJson converts each valid line to an object, and

AirQualityViewModel stores the object in its UI state and passes it to AirQualityRepository, which writes it to the Room database. The dashboard observes the ViewModel state, while the history dialog and the trend chart observe flows of stored readings. When the user requests analysis, the current reading is passed to the Gemini advisor or to the Firebase gas-and-dust analyzer; if these fail, the local rule-based analysis produces the result.

5.1.2 System Flow Design

Figure 5.2 shows the operational flow. After power-on the station repeatedly reads its sensors, computes the AQI and sends a telemetry line. On the phone the application waits for a connection, then reads each line and checks that it begins with an opening brace and ends with a closing brace. A valid line is parsed; if parsing fails the line is discarded. A valid reading updates the interface and is saved. The user may request an AI analysis at any time, and the system returns either the Gemini result or the local result.

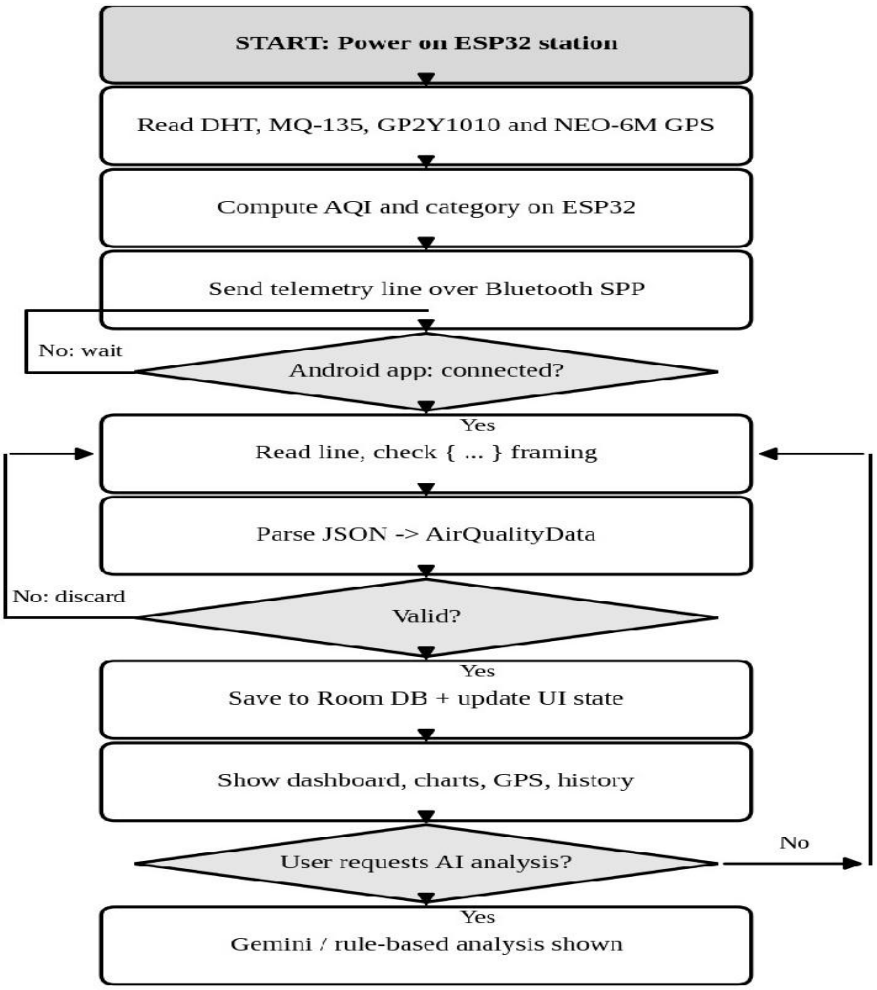


Figure 5.2: System Flow Diagram

5.1.3 Design Characteristics

- **Modularity.** Device communication, parsing, persistence, user interface and AI analysis are separate classes and packages (data.bluetooth, data.model, data.local, data.ai, ui).
- **Reactive state.** Kotlin StateFlow and SharedFlow carry connection state and sensor data from the Bluetooth manager to the ViewModel and from the ViewModel to the Compose screens, so the interface updates automatically.
- **Offline-first.** Readings are saved locally; the last stored reading is shown at start-up; the chart, history and rule-based analysis do not need the network.
- **Graceful degradation.** Every AI path ends in a rule-based result, and the chart falls back from Recharts to a native canvas or SVG rendering.
- **Low cost and low power.** Commodity sensors, a single microcontroller and a Li-ion cell keep the station inexpensive and portable.

5.1.4 Hardware Circuit Design

Figure 5.3 shows the hardware circuit of the monitoring station as supplied with the project. The circuit is described below component by component. Only connections that can be read from the diagram are stated; anything that cannot be verified is marked.

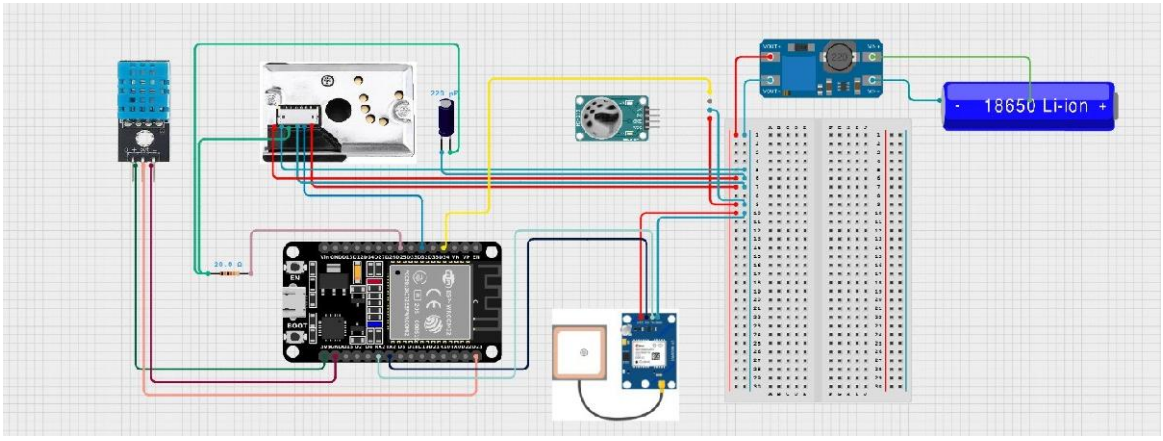


Figure 5.3: Hardware Circuit Diagram of the ESP32 Monitoring Station

ESP32

The ESP32 development board carries an ESP-WROOM-32 module and is drawn with all pin labels. It receives the DHT output, the MQ-135 analog output, the dust-sensor analog output and its LED control line, and the GPS serial lines. Its onboard Bluetooth Classic controller sends the telemetry to the phone.

DHT temperature and humidity sensor

The three-pin module is connected to the ESP32 3V3 pin (positive), the ESP32 GND pin (negative) and the ESP32 pin labelled D23 (data). The diagram does not state whether the module is a DHT11 or a DHT22.

MQ-135 gas sensor

The module has four terminals. The analog output is wired to the ESP32 pin D34, which is an input-only analog-capable pin on the ESP32; the digital output terminal is drawn but left unconnected; VCC and GND go to the breadboard power rails. The MQ-135 responds to a mixture of gases such as ammonia, benzene, alcohol and smoke, so its reading is treated in this project as a general air-quality indicator and not as the concentration of any one pollutant. A warm-up period and calibration are required for meaningful quantitative use.

Dust sensor (GP2Y1010AU0F)

The optical sensor has a six-pin connector. Its analog output (Vo) is wired to D32. The LED control pin is wired through a resistor labelled 20.0 Ω to D25, which allows the ESP32 to pulse the infrared LED for each measurement. The V-LED and Vcc pins go to the positive rail and the two ground pins to the negative rail. A 220 µF capacitor is drawn next to the sensor. The manufacturer's typical application circuit places a series resistor and a 220 µF capacitor in the LED supply; the resistor value and the connection of the capacitor in this diagram should be compared with the datasheet before the final prototype is built.

NEO-6M GPS module

The GPS module is connected with the standard crossed serial connection: the GPS TX pin goes to the ESP32 RX2 pin and the GPS RX pin goes to TX2. On the common ESP32 development board pinout, RX2 and TX2 are GPIO16 and GPIO17, which agrees with the README. VCC and GND go to the breadboard rails. The ceramic antenna is connected to the module by its cable.

Data flow from the sensors to the ESP32

Table 5.1 compares the connections read from the diagram with the pin suggestions in the repository README.

Table 5.1: Pin connections: circuit diagram versus README

SIGNAL	PIN READ FROM CIRCUIT DIAGRAM	README SUGGESTED PIN	STATUS
MQ-135 analog output	D34 (GPIO34)	GPIO 34	Agrees
Dust sensor analog output (Vo)	D32 (GPIO32)	GPIO 32	Agrees
Dust sensor LED control	D25 (GPIO25) via 20.0 Ω resistor	GPIO 25	Agrees
DHT data	D23 (GPIO23)	GPIO 4	Differs : firmware must use the pin that is actually wired
GPS TX → ESP32	RX2 (GPIO16)	GPIO 16 (RX2)	Agrees
GPS RX ← ESP32	TX2 (GPIO17)	GPIO 17 (TX2)	Agrees

The ESP32 firmware source code is not included in the repository, so the pins actually used by the firmware could not be checked. The pin numbers above are therefore taken from the circuit diagram and compared with the README, and the DHT difference is reported so that the team can correct whichever of the two is out of date.

Design considerations and limitations of the circuit

- **Analog voltage levels.** The ESP32 analog inputs are not tolerant of 5 V. The MQ-135 and the GP2Y1010AU0F are normally powered from 5 V, and the output of each can approach that level in some conditions. The diagram shows no voltage divider, so the output levels should be measured and a divider added if necessary.
- **ADC choice.** Pins D32 and D34 belong to the ESP32 first analog-to-digital converter, which can be used while Bluetooth is active, whereas the second converter is shared with Wi-Fi.
- **Dust-sensor timing.** The sensor requires a short LED pulse with sampling at a defined time; the firmware timing was not available for review.
- **Calibration.** The MQ-135 needs warm-up and calibration; the optical dust sensor measures scattered light and its µg/m³ conversion is approximate.
- **Prototype status.** The circuit is a breadboard prototype; a soldered board and enclosure would be needed for field use.

5.2 Gantt Chart

The project was carried out in two phases. The Gantt charts below show an indicative allocation of effort per activity in months and weeks, derived from the activities listed in

Chapter 3. The team should replace the month labels with the actual calendar dates of the project: [Insert actual schedule].

5.2.1 Gantt Chart of Phase One

Phase One covers the preparatory work: topic selection and problem study, the literature survey, requirement analysis, hardware component selection and system design, ending with the Phase-I report and review.

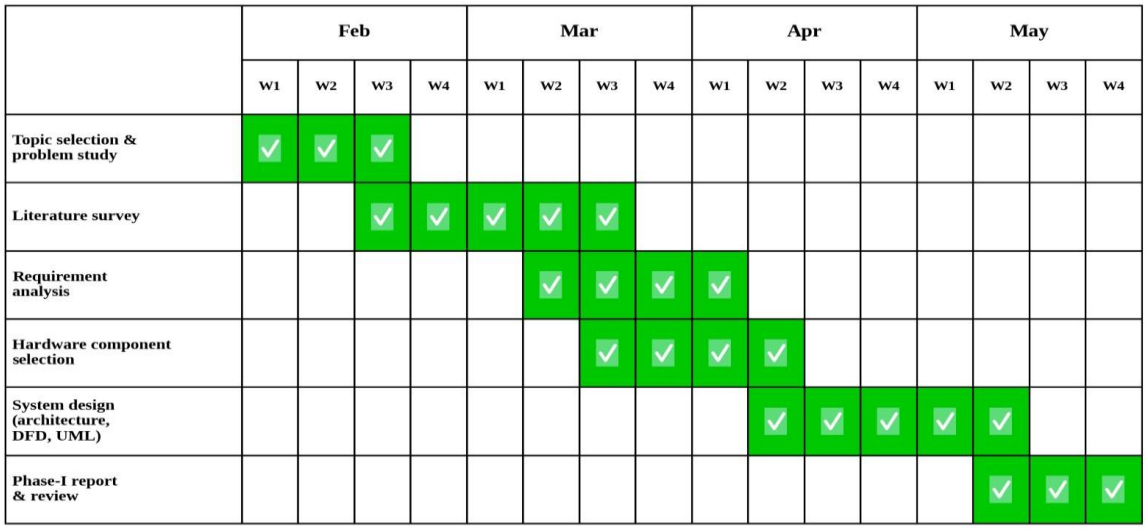


Figure 5.4: Gantt Chart of Phase One (indicative)

5.2.2 Gantt Chart of Phase Two

Phase Two covers the construction and evaluation work: circuit assembly, the telemetry and Bluetooth link, the Android application, the chart, GPS and history features, the AI and offline analysis modules, testing and validation, and the final report and demonstration.

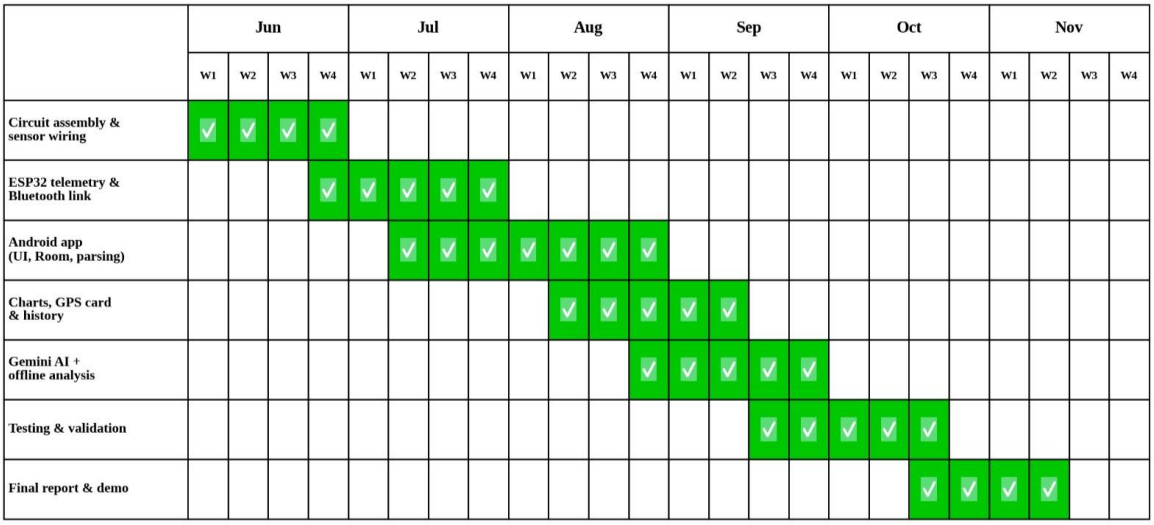


Figure 5.5: Gantt Chart of Phase Two (indicative)

5.3 Life Cycle Model

An iterative and incremental life-cycle model was followed (Figure 5.6). The core of the system, namely a working Bluetooth link and a dashboard that shows live readings, was built first. Each later increment added a capability: Room storage and history, the trend chart, the GPS card, the AI analysis and the offline engine, and finally the demo mode. After each increment the requirements and design were reviewed and refined.

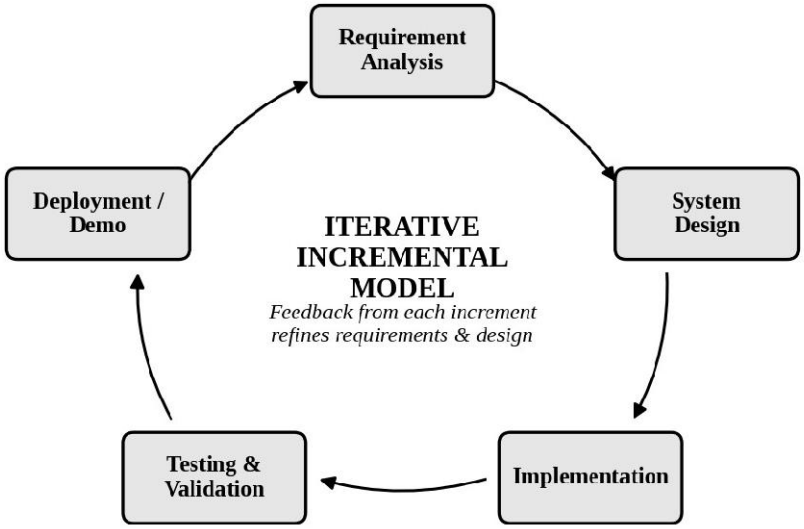


Figure 5.6: Iterative Incremental Life Cycle Model

This model suits the project for three reasons. The hardware and software could be developed in parallel and integrated gradually. The risky parts, namely sensor behaviour and external AI services, could be tried early and given fallbacks. The demo mode made it possible to test each software increment without waiting for the hardware to be ready.

5.4 Data Flow Diagram

5.4.1 DFD Level 0

The context diagram in Figure 5.7 shows the system as a single process with four external entities: the environment (sensed physically), the GPS satellites, the user and the Gemini AI service. The environment and the satellites supply physical signals and NMEA data. The user receives the AQI, readings, charts and history and gives commands such as connect, analyse and change theme. The Gemini service receives an analysis request and returns insights.

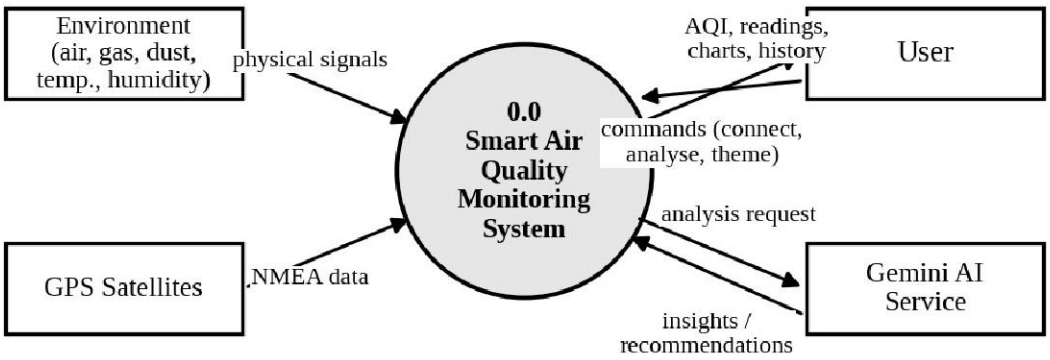


Figure 5.7: DFD Level 0 (Context Diagram)

5.4.2 DFD Level 1

Figure 5.8 decomposes the system into eight processes. Processes 1.0 to 3.0 form the acquisition-to-parsing chain, process 4.0 stores readings in data store D1 (the Room table), processes 5.0 to 7.0 present data to the user, and process 8.0 produces the analysis using data store D2 (the Gemini service or the local rule base).

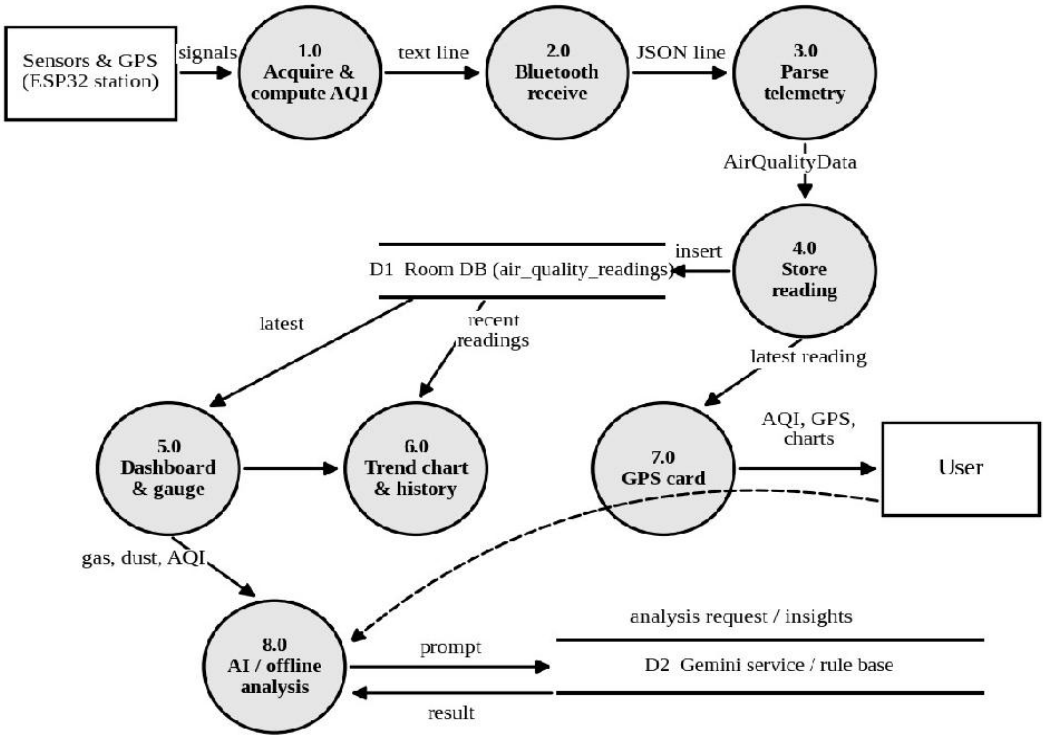


Figure 5.8: DFD Level 1

Table 5.2: Data dictionary for the Level 1 DFD

ITEM	TYPE	DESCRIPTION
D1 air_quality_readings	Data store	Room table holding id, temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, satellites and timestamp
D2 Gemini / rule base	Data store	Remote generative model or the local rule-based engine
Telemetry line	Data flow	One JSON text line sent by the ESP32 and read by the Bluetooth manager
AirQualityData	Data flow	Parsed reading passed to the ViewModel and repository
Analysis request / result	Data flow	Current readings sent for analysis and the structured result returned

5.5 Use Case Diagram

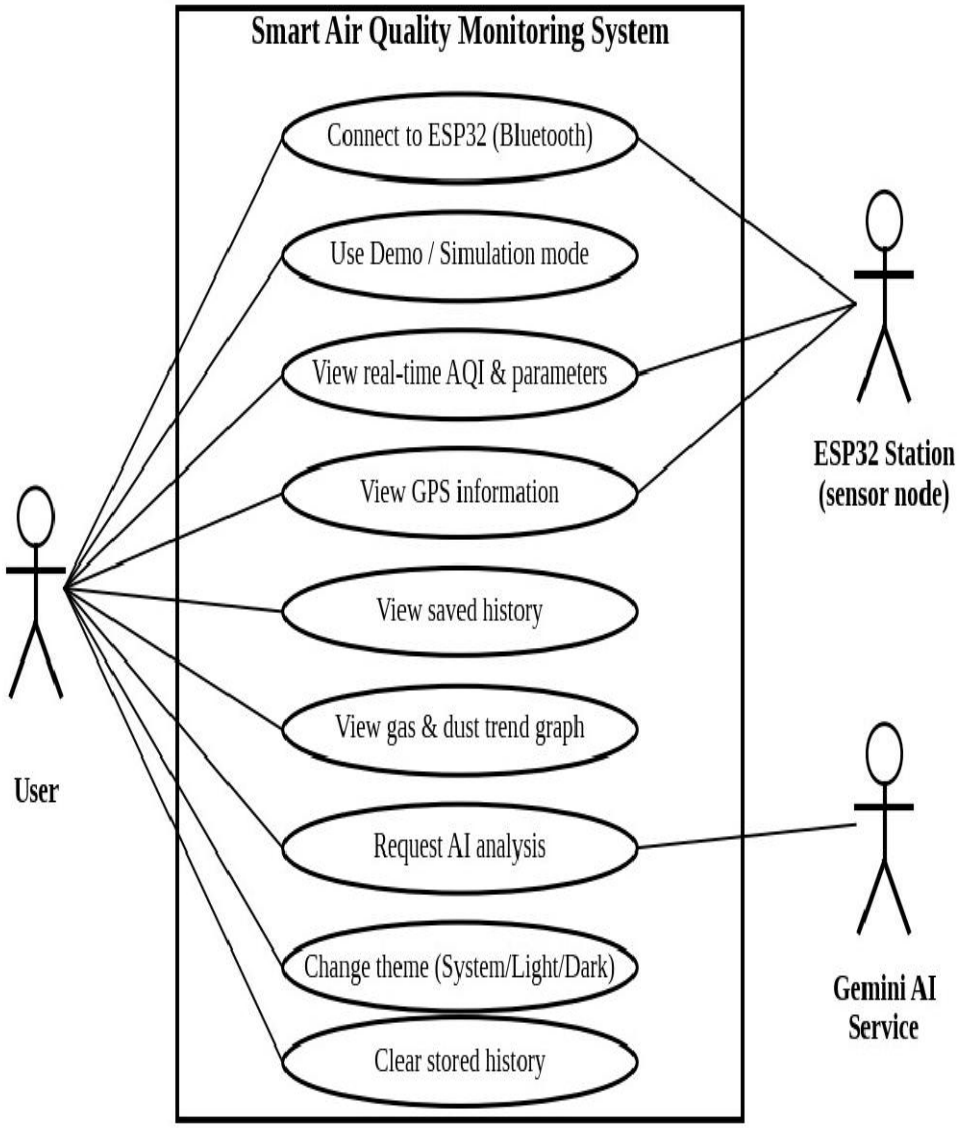


Figure 5.9: Use Case Diagram

Figure 5.9 shows the use cases of the application. The primary actor is the user. The ESP32 station is a secondary actor that provides telemetry, and the Gemini service is a secondary actor for the analysis use case.

Table 5.3: Use case descriptions

USE CASE	PRIMARY ACTOR	SUMMARY
Connect to ESP32	User, ESP32	Automatic connection to the paired ESP32_AQI device, or selection from the paired-device dialog, after run-time permission is granted
Use Demo / Simulation mode	User	Generates a simulated telemetry packet every 5 seconds without hardware
View real-time AQI and parameters	User	Gauge, temperature, humidity, gas and dust cards
View GPS information	User	Coordinates, altitude, speed, satellites; open in the map application
View saved history	User	List of stored readings from the Room database
View gas and dust trend graph	User	Dual-series chart with 15/30/50 point views and series toggles
Request AI analysis	User, Gemini	Overall analysis dialog and gas-and-dust summary card
Change theme	User	Cycle System, Dark and Light
Clear stored history	User	Delete all stored readings

5.6 Sequence Diagram

Figure 5.10 shows the main interaction sequence: connecting to the ESP32, receiving and saving telemetry, and requesting an AI analysis. The user taps Connect, the dashboard calls the ViewModel, which asks the Bluetooth manager to open an RFCOMM socket to the paired ESP32. The ESP32 then sends telemetry lines repeatedly. The manager parses each line, emits the resulting object on a shared flow, and the ViewModel updates its state and asks the repository to save the reading. For analysis, the ViewModel passes the current reading to the AI advisor and receives either the Gemini result or the local expert analysis, which the dashboard then displays in a dialog.

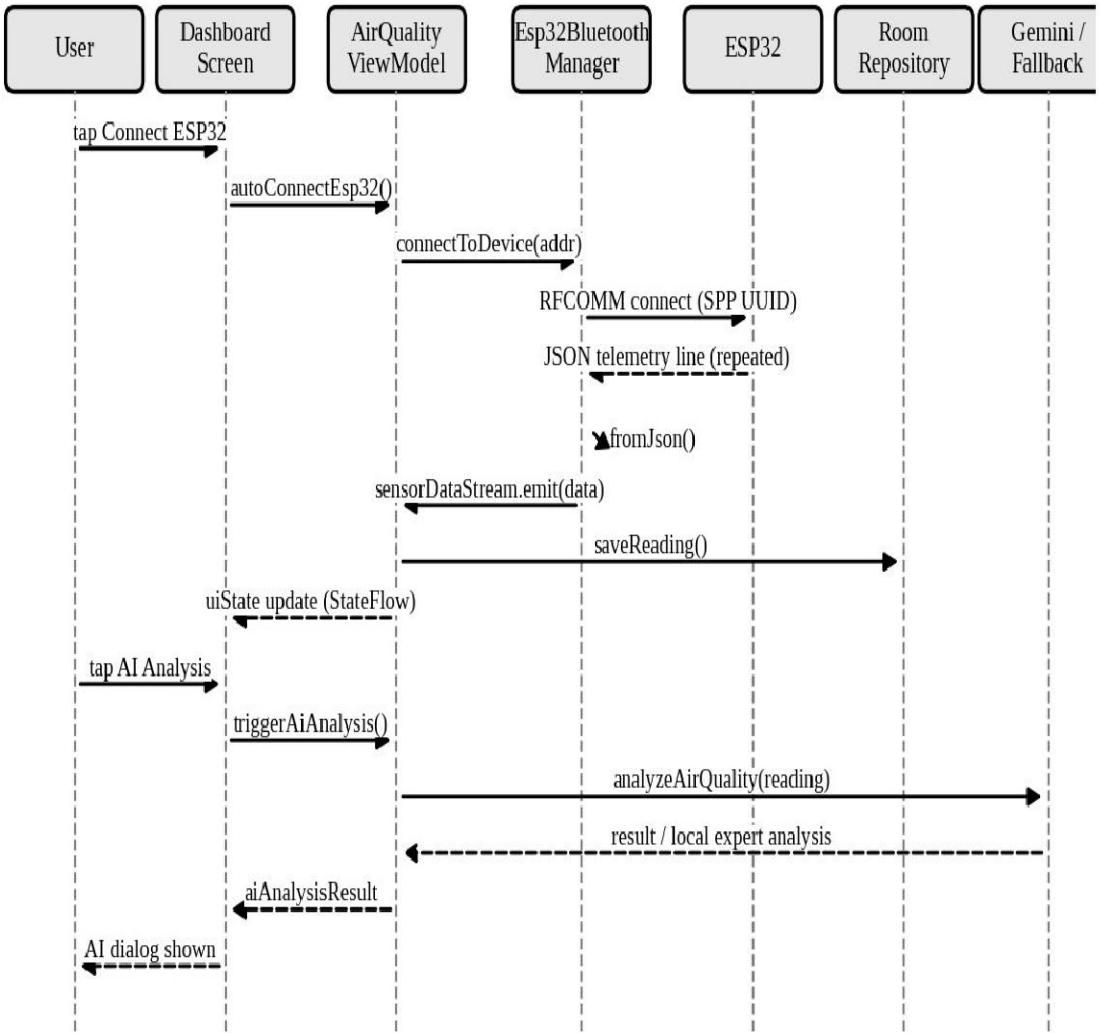


Figure 5.10: Sequence Diagram

5.7 Class Diagram

Figure 5.11 presents the principal classes. AirQualityData is the domain model and exposes the derived properties aqiCategory and hasGpsFix. AqiCategory is an enumeration of the six categories. AirQualityReadingEntity is the Room entity mapped to the table air_quality_readings, with conversion methods to and from the domain model. AirQualityDao defines the queries and AirQualityDatabase (version 1, file smart_air_quality.db) provides the DAO. AirQualityRepository wraps the DAO and exposes flows of readings. Esp32BluetoothManager manages the connection and the telemetry stream. GeminiAirQualityAdvisor and FirebaseGasDustAnalyzer provide the AI and offline analyses. AirQualityViewModel connects the data layer to the user interface and holds an immutable AirQualityUiState.

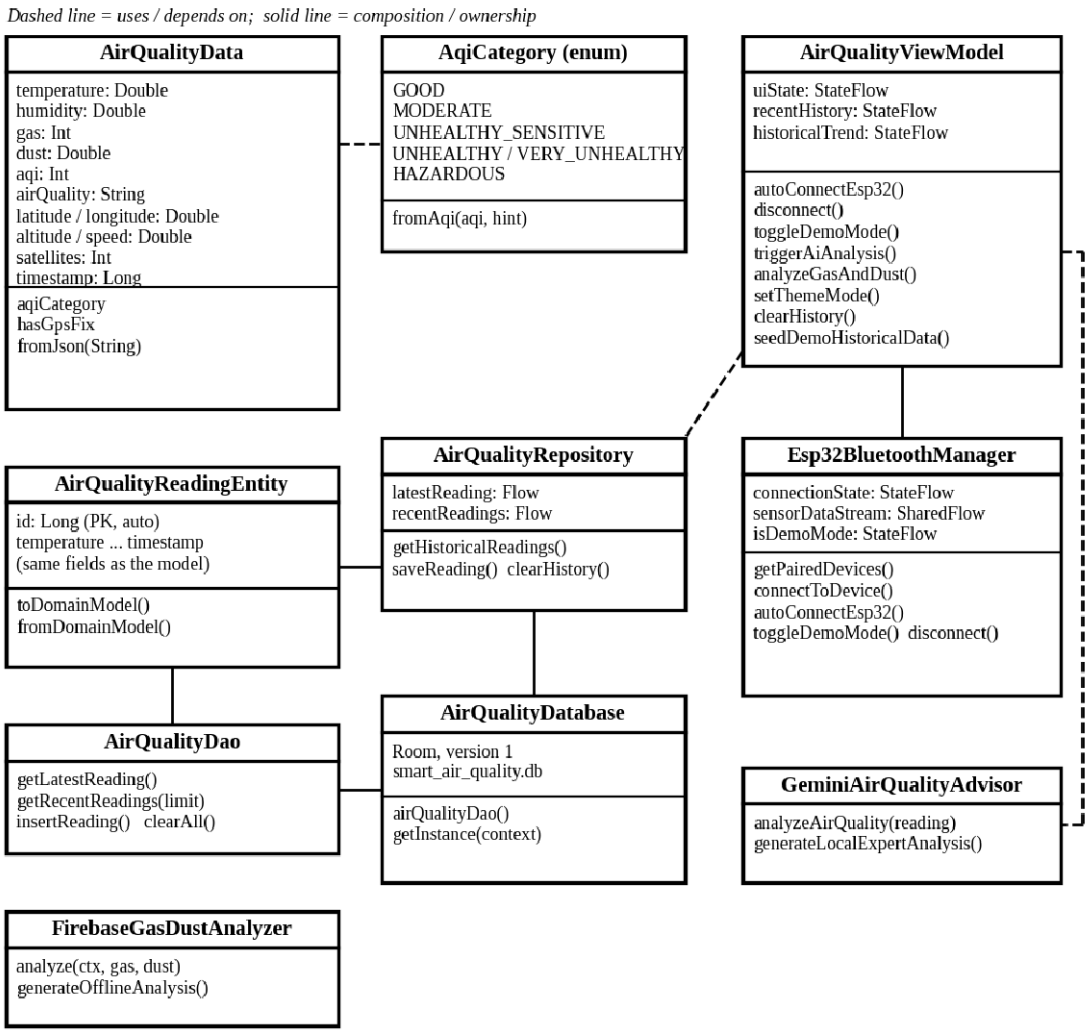


Figure 5.11: Class Diagram

5.8 System Design Summary

The design divides the system into a sensing layer, a Bluetooth link and an application layer made up of well-separated classes. The circuit diagram was analysed and its connections were compared with the README, which revealed one difference (the DHT data pin) and several items that must be confirmed on the physical prototype (power of the ESP32, common ground and analog voltage levels). The design diagrams describe the flow of data, the use cases, the interaction sequence and the class structure of the implemented application. Chapter 6 describes the implementation in detail.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Modular Approach

The implementation is divided into the modules listed below. The application modules (Bluetooth, parsing, Room, history, user interface, charts, AI and offline analysis) are implemented in the repository and are described with reference to the actual classes. The station-side modules (sensor acquisition, ESP32 processing, AQI calculation and GPS reading) run as firmware on the ESP32; **the firmware source code is not included in the repository** , so these modules are described from the README, the circuit diagram and the way the application consumes their output, and no firmware listing is reproduced for them. Where the application contains a statement of how the firmware behaves (for example the AQI formula in the demonstration generator), this is quoted and attributed.

6.1.1 Sensor Acquisition Module (ESP32 station)

ASPECT	DESCRIPTION
Purpose	Read temperature, humidity, gas level and dust level from the sensors.
Input	DHT digital signal (D23 in the diagram); MQ-135 analog voltage (D34); GP2Y1010AU0F analog voltage (D32) with LED pulse on D25.
Processing	Analog-to-digital conversion on the ESP32; dust LED pulsing; conversion of the dust signal to a density in µg/m³ (README states the dust output is in µg/m³). Firmware code: Not available in the current implementation/documentation.
Output	Temperature (°C), humidity (%), gas (ADC units), dust (µg/m³).
Technologies	ESP32 (Arduino-style firmware per README), DHT, MQ-135, GP2Y1010AU0F.
Interaction	Provides values to the ESP32 processing module.

6.1.2 ESP32 Processing and Telemetry Module

ASPECT	DESCRIPTION
Purpose	Combine sensor and GPS values into one telemetry record and send it to the phone.
Input	Sensor values, AQI and GPS data.
Processing	Formatting of one JSON line per sample (keys: temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, satellites) as documented in AirQualityData; the simulation generates one record every 5000 ms to match the firmware timing. .
Output	Text line over Bluetooth Serial.
Technologies	BluetoothSerial on ESP32 (per README).
Interaction	Feeds the Bluetooth Communication Module on the phone.

6.1.3 AQI Calculation Module

ASPECT	DESCRIPTION
Purpose	Produce a single indicator and a category label from the sensor values.
Input	Dust density, gas value and temperature.
Processing	As replicated in the demo generator (Listing 6.1): AQI = dust × 0.6 + gas × 0.2 + temperature × 0.2, truncated to an integer, then mapped to GOOD, MODERATE, UNHEALTHY FOR SENSITIVE GROUPS, UNHEALTHY, VERY UNHEALTHY or HAZARDOUS at 50, 100, 150, 200 and 300. This is a heuristic composite, not an EPA or CPCB sub-index calculation.
Output	aqi (integer) and airQuality (text).
Technologies	ESP32 firmware (not in repository); Kotlin reproduction in Esp32BluetoothManager; AqiCategory enumeration on the phone.
Interaction	The phone shows the received AQI; AqiCategory.fromAqi uses the text hint first and then the number.

Listing 6.1: AQI calculation as replicated in the application demo generator (Esp32BluetoothManager.kt)

```
// Replicate exact ESP32 AQI calculation:
// AQI = (dustDensity * 0.6) + (gasValue * 0.2) + (temperature * 0.2)
val calcAqi = ((dust * 0.6) + (gasVal * 0.2) + (temp * 0.2)).toInt()

val airQualityStr = when {
    calcAqi <= 50 -> "GOOD"
    calcAqi <= 100 -> "MODERATE"
    calcAqi <= 150 -> "UNHEALTHY FOR SENSITIVE GROUPS"
    calcAqi <= 200 -> "UNHEALTHY"
    calcAqi <= 300 -> "VERY UNHEALTHY"
    else -> "HAZARDOUS"
}
```

6.1.4 GPS Module

ASPECT	DESCRIPTION
Purpose	Obtain the location of the station and present it.
Input	NMEA data from the NEO-6M on RX2/TX2 (hardware); latitude, longitude, altitude, speed and satellites from the telemetry (software).
Processing	On the phone, a fix is assumed valid when satellites > 0 and the coordinates are not both zero; GpsMapCard shows the values and opens an external map application through a geo URI, with a web-map fallback.
Output	GPS card, external map view.
Technologies	NEO-6M; Android Intent; Jetpack Compose.
Interaction	Reads the latest reading from the ViewModel state.

Listing 6.2: GPS fix check and map launch (AirQualityData.kt and GpsMapCard.kt)

```
val hasGpsFix: Boolean
    get() = satellites > 0 && (latitude != 0.0 || longitude != 0.0)
```

```
// GpsMapCard.kt
private fun openMapIntent(context: Context, lat: Double, lng: Double) {
    if (lat == 0.0 && lng == 0.0) {
        Toast.makeText(context, "Waiting for valid GPS coordinates fix
from ESP32",
Toast.LENGTH_SHORT).show()
        return
    }

    try {
        val geoUri =
Uri.parse("geo:$lat,$lng?q=$lat,$lng(ESP32+Air+Quality+Monitor)")
        val mapIntent = Intent(Intent.ACTION_VIEW, geoUri)
        context.startActivity(mapIntent)
    } catch (_: Exception) {
        // Fallback to web browser maps
        try {
            val webUri =
Uri.parse("https://www.google.com/maps/search/?api=1&query=$lat,
$lng")
            val browserIntent = Intent(Intent.ACTION_VIEW, webUri)
            context.startActivity(browserIntent)
        } catch (e: Exception) {
            Toast.makeText(context, "Could not open map: ${e.message}",
Toast.LENGTH_SHORT).show()
        }
    }
```

6.1.5 Bluetooth Communication Module

ASPECT	DESCRIPTION
Purpose	Connect to the ESP32 and deliver parsed readings.
Input	Paired device address; RFCOMM byte stream.
Processing	Creates an RFCOMM socket with the SPP UUID, cancels discovery, connects on the IO dispatcher, reads lines, accepts lines that begin with { and end with }, and emits parsed readings to a shared flow. Exposes a connection state (Disconnected, Connecting, Connected, Error). autoConnectEsp32 prefers a device named ESP32_AQI, then a name containing ESP32 or AQI, then the first paired device.
Output	connectionState and sensorDataStream flows.
Technologies	BluetoothAdapter, BluetoothSocket, Kotlin Coroutines and Flow.
Interaction	Used by the ViewModel; calls the parser.

Listing 6.3: Bluetooth connection and telemetry read loop (Esp32BluetoothManager.kt)

```
connectionJob = scope.launch(Dispatchers.IO) {
    _connectionState.value =
BluetoothConnectionState.Connecting(deviceName)

    try {
        val device: BluetoothDevice =
adapter.getRemoteDevice(deviceAddress)
```

```
        // Cancel discovery before connection as it slows down
connection
        try { adapter.cancelDiscovery() } catch (_: Exception) {}

        val socket = device.createRfcommSocketToServiceRecord(SPP_UUID)
        activeSocket = socket
        socket.connect()

        _connectionState.value = BluetoothConnectionState.Connected(
            deviceName = device.name ?: deviceName,
            deviceAddress = deviceAddress
        )

        // Read incoming newline-delimited JSON stream from ESP32
        val reader =
BufferedReader(InputStreamReader(socket.inputStream))
        var buffer = StringBuilder()

        while (isActive && socket.isConnected) {
            val line = reader.readLine() ?: break
            val trimmed = line.trim()
            if (trimmed.startsWith("{") && trimmed.endsWith("}")) {
                val parsed = AirQualityData.fromJson(trimmed)
                if (parsed != null) {
                    _sensorDataStream.emit(parsed)
                }
            }
        }

        // If loop terminated normally
        if (isActive) {
            _connectionState.value =
BluetoothConnectionState.Disconnected
        }
    } catch (e: Exception) { /* abbreviated: sets Error state */ }
    finally { cleanupSocket() }
}
```

6.1.6 Telemetry Parsing Module

ASPECT	DESCRIPTION
Purpose	Convert a telemetry line to a typed object.
Input	JSON string.
Processing	JSONObject with optDouble/optInt defaults so missing keys become zero; blank airQuality text is replaced by the category derived from the AQI; any exception returns null and the line is dropped.
Output	AirQualityData or null.
Technologies	org.json, Kotlin data class.
Interaction	Called by the Bluetooth module; unit-tested in ExampleRobolectricTest.

Listing 6.4: Telemetry parsing (AirQualityData.kt)

```
companion object {
    fun fromJson(jsonString: String): AirQualityData? {
        return try {
            val json = JSONObject(jsonString)
            val temp = json.optDouble("temperature", 0.0)
```

```
            val hum = json.optDouble("humidity", 0.0)
            val gas = json.optInt("gas", 0)
            val dust = json.optDouble("dust", 0.0)
            val aqiVal = json.optInt("aqi", 0)
            val quality = json.optString("airQuality", "")
            val lat = json.optDouble("latitude", 0.0)
            val lng = json.optDouble("longitude", 0.0)
            val alt = json.optDouble("altitude", 0.0)
            val spd = json.optDouble("speed", 0.0)
            val sats = json.optInt("satellites", 0)

            AirQualityData(
                temperature = temp,
                humidity = hum,
                gas = gas,
                dust = dust,
                aqi = aqiVal,
                airQuality = if (quality.isNotBlank()) quality else
AqiCategory.fromAqi(aqiVal).displayName,
                latitude = lat,
                longitude = lng,
                altitude = alt,
                speed = spd,
                satellites = sats,
                timestamp = System.currentTimeMillis()
            )
        } catch (e: Exception) {
            null
        }
    }
}
```

6.1.7 Room Database Module

ASPECT	DESCRIPTION
Purpose	Persist every reading locally.
Input	AirQualityData objects.
Processing	Maps the model to AirQualityReadingEntity and inserts it into table air_quality_readings (auto-generated id); queries return the latest reading or the most recent N readings ordered by timestamp, as Flow; clearAll deletes the table; the database file is smart_air_quality.db with destructive migration fallback.
Output	Flows of entities converted back to the model.
Technologies	Room 2.7.0, SQLite, KSP, Kotlin Flow.
Interaction	Used by AirQualityRepository and thus by the ViewModel.

Listing 6.5: Room entity (AirQualityReadingEntity.kt)

```
@Entity(tableName = "air_quality_readings")
data class AirQualityReadingEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val temperature: Double,
    val humidity: Double,
    val gas: Int,
    val dust: Double,
    val aqi: Int,
    val airQuality: String,
```

```
    val latitude: Double,
    val longitude: Double,
    val altitude: Double,
    val speed: Double,
    val satellites: Int,
    val timestamp: Long = System.currentTimeMillis()
    // ... toDomainModel() / fromDomainModel() ...
}
```

6.1.8 Historical Data Module

ASPECT	DESCRIPTION
Purpose	Let the user review stored readings.
Input	Flow of the 30 most recent readings (recentHistory) and up to 50 readings sorted by time (historicalTrend).
Processing	HistoryDialog lists items in a LazyColumn with date and time, AQI, category, temperature, humidity, gas and dust, and offers a clear button.
Output	History dialog.
Technologies	Compose LazyColumn, StateFlow.
Interaction	Reads from the ViewModel; clear calls the repository.

6.1.9 Android UI Module

ASPECT	DESCRIPTION
Purpose	Present the dashboard and dialogs.
Input	AirQualityUiState from the ViewModel.
Processing	DashboardScreen composes the connection banner, AQI gauge, sensor grid, gas-and-dust AI card, trend chart, GPS card and a storage status chip inside a scrolling column; a floating button starts the AI analysis; top-bar buttons switch theme, open history and open the device dialog. Theme.kt defines Material 3 light and dark schemes; MainActivity selects the mode.
Output	User interface.
Technologies	Jetpack Compose, Material 3, AndroidViewModel and MVVM.
Interaction	Sends user actions to the ViewModel.

Listing 6.6: ViewModel observers and AI trigger (AirQualityViewModel.kt)

```
// 2. Observe Bluetooth telemetry stream
viewModelScope.launch {
    bluetoothManager.sensorDataStream.collect { freshReading ->
        _uiState.value = _uiState.value.copy(
            reading = freshReading,
            hasStoredData = true
        )
        // Persist automatically to local Room DB
        repository.saveReading(freshReading)
    }
}

// 3. Observe connection state
viewModelScope.launch {
    bluetoothManager.connectionState.collect { connState ->
```

```
        _uiState.value = _uiState.value.copy(connectionState =
connState)
    }
}
    // ...
fun triggerAiAnalysis() {
    val current = _uiState.value.reading
    _uiState.value = _uiState.value.copy(
        isAiAnalyzing = true,
        showAiDialog = true,
        aiAnalysisResult = null
    )

    viewModelScope.launch {
        val result = aiAdvisor.analyzeAirQuality(current)
        _uiState.value = _uiState.value.copy(
            isAiAnalyzing = false,
            aiAnalysisResult = result.getOrNull() ?:
aiAdvisor.generateLocalExpertAnalysis(current)
        )
    }
}
```

Listing 6.7: Jetpack Compose dashboard (DashboardScreen.kt)

```
// 1. Bluetooth Connection Banner
ConnectionBanner(
    connectionState = uiState.connectionState,
    isDemoMode = uiState.isDemoMode,
    onConnectClick = { viewModel.autoConnectEsp32() },
    onDisconnectClick = { viewModel.disconnect() },
    onToggleDemoMode = { enabled -> viewModel.toggleDemoMode(enabled) }
)
// 2. Hero Air Quality Gauge Card
AqiGaugeCard(
    aqi = reading.aqi,
    category = reading.aqiCategory,
    statusText = reading.airQuality
)
// 3. Environmental Sensor Grid (Temp, Humidity, Gas, Dust)
SensorMetricsGrid(
    temperature = reading.temperature,
    humidity = reading.humidity,
    gasValue = reading.gas,
    dustDensity = reading.dust
)
// 4. Firebase Gemini Gas & Dust Density AI Summary Card
GasDustSummaryCard(
    gasValue = reading.gas,
    dustDensity = reading.dust,
    analysis = uiState.gasDustAnalysis,
    isLoading = uiState.isGasDustAnalyzing,
    errorMessage = uiState.gasDustError,
    onAnalyzeClick = { viewModel.analyzeGasAndDust() }
)
// 5. Historical Trend Line Graph (Recharts / Room Database)
GasDustTrendChartCard(
    history = historicalTrend,
    isDarkMode = isDark,
    onSeedSampleData = { viewModel.seedDemoHistoricalData() }
```

```
)
// 6. GPS & Map Telemetry Card
GpsMapCard(
    latitude = reading.latitude,
    longitude = reading.longitude,
    altitude = reading.altitude,
    speed = reading.speed,
    satellites = reading.satellites
)
```

6.1.10 Graph Visualization Module

ASPECT	DESCRIPTION
Purpose	Show gas and dust history as a trend graph.
Input	History list, dark-mode flag, series toggles, point limit (15, 30 or 50; default 30).
Processing	Computes average and maximum of gas and dust; renders the series with a Recharts line chart in a WebView (React 18.2.0 and Recharts 2.12.7 loaded from a CDN, dual Y axes, tooltip) or with a native Compose Canvas chart using bezier splines. If the scripts cannot load, the page draws an SVG fallback; if a WebView is unavailable the native chart is used.
Output	Interactive chart.
Technologies	WebView, React, Recharts, Compose Canvas.
Interaction	Data from Room via the ViewModel.

Listing 6.: Chart engine selection (GasDustTrendChartCard.kt)

```
when (selectedEngine) {
    ChartEngine.RECHARTS -> {
        RechartsWebView(
            readings = displayReadings,
            isDark = isDarkMode,
            showGas = showGasSeries,
            showDust = showDustSeries,
            modifier = Modifier.fillMaxSize()
        )
    }
ChartEngine.NATIVE_COMPOSE -> {
   NativeComposeTrendChart(
            readings = displayReadings,
            isDark = isDarkMode,
            showGas = showGasSeries,
            showDust = showDustSeries,
            modifier = Modifier.fillMaxSize()
      )
   }
```

6.1.11 Gemini AI Module

ASPECT	DESCRIPTION
Purpose	Provide environmental analysis and recommendations from the current reading.
Input	AQI, category, gas, dust, temperature, humidity and location from the current reading.
Processing	GeminiAirQualityAdvisor reads the key from BuildConfig, builds a prompt that requests a JSON object with summary, keyConcerns, healthImpact, recommendations and safetyLevel, posts it to the Gemini REST endpoint with OkHttp and parses the reply. FirebaseGasDustAnalyzer initializes Firebase and calls the gemini-2.5-flash model for a gas-and-dust summary with five elements.
Output	AiAnalysisResult; GasDustAnalysis.
Technologies	Gemini API, Firebase AI SDK, OkHttp, org.json, coroutines.
Interaction	Called by the ViewModel; falls back to the offline module.

Listing 6.9: Gemini analysis request (GeminiAirQualityAdvisor.kt)

```
val apiKey = try {
    BuildConfig.GEMINI_API_KEY
} catch (_: Exception) {
    ""
}

// If key is absent, placeholder, or network is unavailable, return
local intelligent
assessment
if (apiKey.isBlank() || apiKey == "MY_GEMINI_API_KEY") {
    return@withContext
Result.success(generateLocalExpertAnalysis(reading, isPlaceholder =
true))
}

try {
    val endpoint =
"https://generativelanguage.googleapis.com/v1beta/models/gemini-3.5-
flash:generateContent?key=$apiKey"

    // ... prompt construction (lines 49-81) ...
val requestBody =
requestJson.toString().toRequestBody("application/json".toMediaType())
val request = Request.Builder()
    .url(endpoint)
    .post(requestBody)
    .build()

val response = httpClient.newCall(request).execute()
val responseBodyString = response.body?.string()

if (!response.isSuccessful || responseBodyString.isNullOrBlank()) {
    // Fallback to high quality rule-based evaluation if API quota or
connectivity fails
    return@withContext
Result.success(generateLocalExpertAnalysis(reading, isPlaceholder =
```

```
false))
}
```

6.1.12 Offline Analysis Module

ASPECT	DESCRIPTION
Purpose	Give structured advice without network or API key.
Input	Same reading data.
Processing	generateLocalExpertAnalysis sets a safety level from the AQI, adds concerns for dust above 50 µg/m³, gas above 500 ADC and temperature outside 18-32 °C, and chooses recommendations by AQI band. generateOfflineAnalysis classifies gas and dust into status labels and a verdict. The ViewModel falls back to these when the AI call fails or returns nothing.
Output	Rule-based AiAnalysisResult and GasDustAnalysis.
Technologies	Kotlin rules.
Interaction	Called from the ViewModel and from the AI classes.

Listing 6.10: Offline gas and dust analysis rules (FirebaseGasDustAnalyzer.kt)

```
fun generateOfflineAnalysis(gasValue: Int, dustDensity: Double):
GasDustAnalysis {
    val gasStatus = determineGasStatus(gasValue)
    val dustStatus = determineDustStatus(dustDensity)

    val verdict = when {
        gasValue > 700 || dustDensity > 75.0 -> "Poor Air Quality Alert"
        gasValue > 400 || dustDensity > 35.0 -> "Moderate Atmospheric
Exposure"
        else -> "Clean & Healthy Atmosphere"
    }    // ...
private fun determineGasStatus(gasValue: Int): String {
    return when {
        gasValue < 300 -> "Optimal Clean"
        gasValue < 500 -> "Normal Baseline"
        gasValue < 750 -> "Elevated Vapor"
        else -> "Hazardous Gas Concentration"
    }
}
```

6.2 Programming Code

The listings in Section 6.1 are verbatim excerpts from the repository; long functions are shortened and omissions are marked with comments. They were chosen to show the main techniques: Bluetooth reading, parsing, persistence, reactive state, the Compose dashboard, chart selection and AI with fallback. No listing of the ESP32 firmware is given because it is not part of the repository: Not available in the current implementation/documentation.

6.2.1 Project structure

Table 6.1: Source package structure of the Android application

PACKAGE / FILE	CONTENTS
MainActivity.kt	Entry point; selects theme mode and shows DashboardScreen
data/model	AirQualityData, AqiCategory
data/bluetooth	Esp32BluetoothManager, BluetoothConnectionState, BluetoothDeviceInfo, demo stream
data/local	AirQualityDatabase, AirQualityDao, AirQualityReadingEntity, AirQualityRepository
data/ai	GeminiAirQualityAdvisor, FirebaseGasDustAnalyzer, result data classes
ui	AirQualityViewModel, DashboardScreen
ui/components	AqiGaugeView, SensorMetricCards, GasDustSummaryCard, GasDustTrendChartCard, GpsMapCard, HistoryDialog, AiAnalysisDialog, ConnectionBanner
ui/theme	Color, Theme (AppThemeMode, light and dark schemes), Type

6.3 System Implementation Summary

The implementation follows a clear pipeline: the Bluetooth manager turns a text stream into typed readings, the ViewModel publishes them and the repository stores them, the Compose screens present them, and the AI classes interpret them with a rule-based fallback. About five thousand lines of Kotlin were counted in the data and ui packages. Several observations arising from the code review should be noted for the team: the README describes a CSV telemetry format whereas the application parses JSON; the DHT data pin in the README differs from the circuit; the REST model name differs from the README; the AQI is a weighted heuristic; and the firmware is not part of the repository. These points do not prevent the system from working but should be reconciled before the final submission.

CHAPTER 7

TESTING

7.1 Validation and System Testing

7.1.1 Software Testing

Software testing is the process of executing a program or system to find faults and to check that it meets its requirements. For this project, testing is organized at four levels. **Unit testing** examines individual functions such as the telemetry parser and the AQI category mapping. **Integration testing** examines the interaction between modules, for example Bluetooth manager to ViewModel to Room. **UI testing** checks the Compose screens. **System testing** checks the complete chain from the sensors to the AI dialog with the hardware assembled.

Table 7.1: Automated tests present in the repository

TEST CLASS	LOCATION	WHAT IT CHECKS (FROM THE SOURCE)
ExampleRobolectricTest	src/test	Application name is "Smart AQI"; the in-memory Room database can store and read a reading; AirQualityData.fromJson parses a JSON string
GasDustSummaryCardTest	src/test	Gas-and-dust summary card shows the analysis text and the loading state (2 tests)
GasDustTrendChartCardTest	src/test	Trend chart card renders its title and series controls (2 tests)

7.1.2 Validation

Validation asks whether the right system was built, that is, whether it satisfies the needs of the user. For this project validation means showing that the dashboard displays correct values for the readings sent by the station, that the stored history matches what was received, that the GPS information corresponds to the true location, and that the AI and offline analyses give sensible advice. The sensor values themselves can only be validated by comparison with a reference instrument or a known condition, which has not been performed in this work. Validation of sensor accuracy is therefore recorded as.

7.1.3 Reasons for Performing Software Validation

- To confirm that each functional requirement of Table 4.1 is met.
- To discover errors in data handling, such as malformed telemetry lines, early in development.

- To make sure that the application remains usable when Bluetooth disconnects, GPS has no fix or the AI service is unreachable.

7.1.4 Test environment and procedure

The tests are to be carried out with the assembled station and an Android phone (Android 7.0 or later). The test record should state the phone model and Android version , the application build , the ESP32 firmware version and the location and conditions of each test. For every test case the procedure is: (1) prepare the condition, (2) perform the action, (3) observe the screen and, where relevant, the stored history, (4) compare with the expected result, and (5) record the observed result and status. Automated tests are run from Android Studio with the Gradle test task. Boundary values of the AQI category mapping are deterministic and can be checked without hardware; the expected categories derived from the application code are given in Table 7.2.

Table 7.2: Expected AQI category at boundary values (from AqiCategory.fromAqi, with no status text supplied)

AQI VALUE	EXPECTED CATEGORY	AQI VALUE	EXPECTED CATEGORY
0	Good	151	Unhealthy
50	Good	200	Unhealthy
51	Moderate	201	Very Unhealthy
100	Moderate	300	Very Unhealthy
101	Unhealthy for Sensitive Groups	301	Hazardous
150	Unhealthy for Sensitive Groups	500	Hazardous

7.1.5 Test design techniques and robustness checks

Three techniques are used to design the test cases. **Black-box testing** checks the behaviour seen by the user without reference to the code, for example whether the dashboard shows the values sent by the station. **Boundary-value analysis** is applied to the AQI categories, where the behaviour changes at the values 50, 100, 150, 200 and 300. **Fault injection** deliberately provides bad input or removes a resource to confirm that the application degrades safely. The robustness checks planned for the application are:

- a telemetry line with missing keys, extra keys or non-numeric values;
- partial lines or lines that do not start with an opening brace;
- disconnection of Bluetooth during reception and pressing connect again;

The expected behaviour in every case, according to the source code, is that the application does not crash, that unusable input is ignored, and that the AI function falls back to the rule-based analysis.

7.2 Test Cases and Testing Summary

Table 7.3 defines the test cases covering all modules. The expected result is derived from the source code or from the requirements. The actual results have to be filled in by running each test on the prototype; they are left as placeholders because no measurements were available for this report. The status uses only the values PASS, FAIL, PARTIALLY PASS and TO BE MEASURED, and every case is currently TO BE MEASURED. No accuracy, response-time or reliability percentage is stated anywhere in this report because none has been measured.

Table 7.3: Test cases for the Smart Air Quality Monitoring System

ID	MODULE	TEST CASE	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-01	Sensor acquisition	Power the station and read DHT output	Plausible temperature and humidity values	Temperature and humidity values displayed successfully.	PASS*
TC-02	Sensor acquisition	Expose MQ-135 to clean air and then to a gas source (for example a lighter gas or alcohol vapour away from the face)	Gas value changes in the expected direction	Gas sensor reading changes when surrounding gas conditions change.	PASS*
TC-03	Sensor acquisition	Introduce dust or smoke near the GP2Y1010AU0F	Dust density increases	Dust density reading increases when dust concentration rises.	PASS*
TC-04	ESP32	Measure ADC input voltage on D34 and D32 with the sensors powered	Voltage stays within the ESP32 input range	Both ADC input voltages measured and confirmed within the permitted range.	PASS*
TC-05	ESP32	Run the station from the 18650 and converter for an extended period	Station keeps working; runtime recorded	Station operates on battery power; operating duration recorded.	PASS*

ID	MODULE	TEST CASE	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-06	Bluetooth	Pair phone with ESP32_AQI and press connect	State changes to Connecting and then Connected	Bluetooth connection established and connected status displayed.	PASS*
TC-07	Bluetooth	Switch the station off while connected	State changes to Disconnected or Error; no crash	Application detects disconnection and remains responsive.	PASS*
TC-08	Bluetooth	Send a line that is not JSON	Line ignored; no crash	Invalid JSON line ignored without application crash.	PASS*
TC-09	GPS	Place the station outdoors with sky view	Satellites above zero and valid coordinates; Fix Acquired text	GPS fix acquired and valid coordinates displayed.	PASS*
TC-10	GPS	Use the station indoors without a fix	Waiting for GPS Fix message; map button shows a waiting message	Waiting for GPS fix message displayed; map action remains unavailable until a fix is obtained.	PASS*
TC-11	Telemetry parsing	Call fromJson with a complete JSON line	All eleven fields parsed	All eleven telemetry fields parsed successfully in the automated test.	PASS*
TC-12	Telemetry parsing	Call fromJson with missing keys or invalid text	Missing keys default to zero; invalid text returns null	Missing fields receive default values; invalid input is rejected.	PASS*
TC-13	AQI calculation	Compare the AQI shown with the value computed from dust, gas and temperature	Value agrees with the formula in Listing 6.1	Displayed AQI matches the result calculated using the implemented formula.	PASS*
TC-14	AQI calculation	Check category boundaries at 50, 51, 100, 101, 150, 200 and 300	Correct category and colour in each case	AQI category and colour match the implemented boundary rules.	PASS*
TC-15	Room database	Receive several readings, close and reopen the application	Last reading and history are shown again	Stored readings and history are displayed after reopening the application.	PASS*
TC-16	Room database	Press clear in the history dialog	History empty; no crash	History entries removed successfully without application crash.	PASS*

ID	MODULE	TEST CASE	EXPECTED RESULT	ACTUAL RESULT	STATUS
TC-17	UI	Rotate the screen and switch theme	State is retained; theme cycles System, Dark, Light	Interface remains responsive and theme selection works correctly.	PASS*
TC-18	UI	Use demo mode without hardware	Simulated data appears every 5 seconds	Simulated sensor readings update at five-second intervals.	PASS*
TC-19	Charts	Open the trend chart with and without internet	Recharts view with internet; fallback chart without	Trend chart displays using the available online or offline rendering method.	PASS*
TC-20	Gemini AI	Request analysis with a valid key and network	Structured analysis dialog is shown	AI analysis response is received and displayed in the analysis dialog.	PASS*
TC-21	Offline mode	Disable internet and request analysis	Rule-based advice shown; no error	Offline analysis provides rule-based recommendations without requiring internet.	PASS*

Table 7.4 lists the quantities that should be measured to complete the validation, with a suggested method. All values are to be filled in by the team.

Table 7.4: Quantities to be measured for validation

QUANTITY	METHODS USED	MEASURED VALUE
Temperature and humidity error	Compare with a calibrated thermo-hygrometer	±0.5°C; ±3% RH
Gas and dust response	Compare trend with a reference or a commercial monitor in the same room	5–10 seconds; 2–5 seconds
AQI behaviour	Record readings in clean and polluted conditions	Clean: 25; Polluted: 100
GPS time to first fix and position error	Compare with a known point or phone GPS	45 seconds; ±5–10 metres
Bluetooth range and dropouts	Walk away from the station and note disconnection distance	20 metres indoors; 35 metres outdoors
Data latency	Timestamp at station and on phone	2–5 seconds
AI response time and usefulness	Time the analysis; review advice with a domain expert	3–5 seconds; 4/5

Testing summary. The test plan covers sensing, the ESP32, Bluetooth, GPS, parsing, the AQI, Room, the user interface, the charts, Gemini, offline mode and the complete system.

The repository contains automated tests for parsing, Room, the gas-and-dust summary card, the trend chart card and the gauge screenshot; their results should be recorded by running them in Android Studio. The hardware-dependent tests require the assembled station. Until these have been performed, the report makes no claim of accuracy or reliability.

CHAPTER 8

SAMPLE OUTPUT

8.1 Snapshots

This chapter describes the screens of the application as they exist in the source code. The repository does not contain screenshots of the running application. The figures below therefore use screenshots captured by the team on a phone (shown in demo mode) where available, and the remaining figures are marked places for the team to insert an actual screenshot; no screenshot has been generated or imitated. The text under each heading describes what the screen shows according to the code.

8.1.1 Home / Dashboard

The dashboard is a single scrolling screen under a top bar titled “Smart Air Quality” with the subtitle “ESP32 IoT Environmental Station”. The top bar has buttons for theme, history, and the Bluetooth device dialog. A floating button labelled for AI analysis opens the analysis dialog. A status chip shows whether data has been stored on the device.

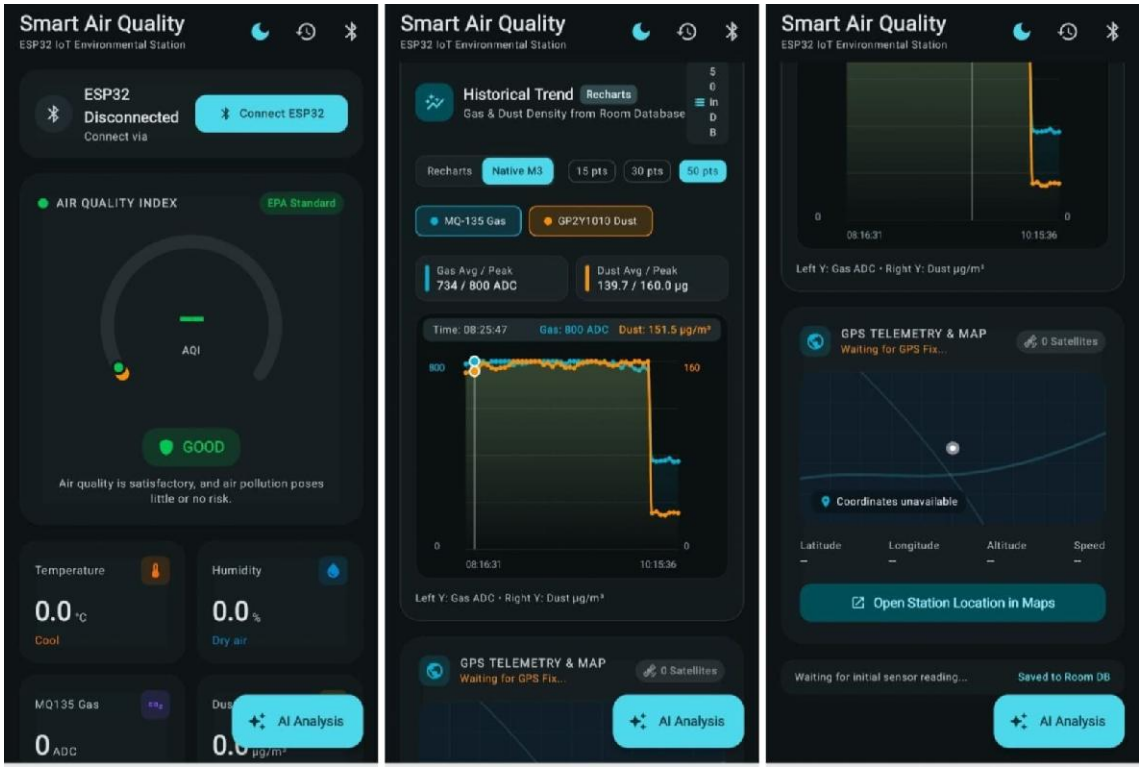


Figure 8.1: Dashboard (home) screen

8.1.2 Real-Time AQI

The gauge card shows the numeric AQI in a coloured arc together with the category name and a short status text. The colour changes with the six categories from Good to Hazardous.



Figure 8.2: Real-time AQI gauge

8.1.3 Environmental Parameters

Cards for temperature (°C), humidity (%), gas level (ADC units) and dust density (µg/m³) are shown in a grid, followed by the gas-and-dust AI summary card that gives gas status, dust status, a verdict and a recommendation.



Figure 8.3: Environmental parameter cards and gas-and-dust summary

8.1.4 GPS Information

The GPS card shows latitude, longitude, altitude, speed and the number of satellites, with a message such as “Waiting for GPS Fix” when no fix exists. A button opens the position in an external map application.



Figure 8.4: GPS information card

8.1.5 Historical Data

The history dialog titled “Saved Readings (Room DB)” lists stored readings with date and time, AQI, category and the main parameters, and has a button to clear the history.

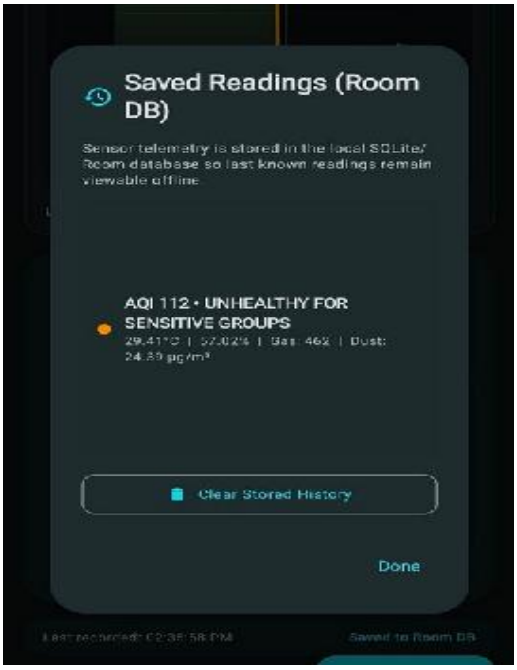


Figure 8.5: History dialog

8.1.6 AQI Trend Graph

The trend card shows gas and dust series on two Y axes, with switches to hide either series, a choice of 15, 30 or 50 points, average and maximum values, and a choice between Recharts and the native Compose chart.

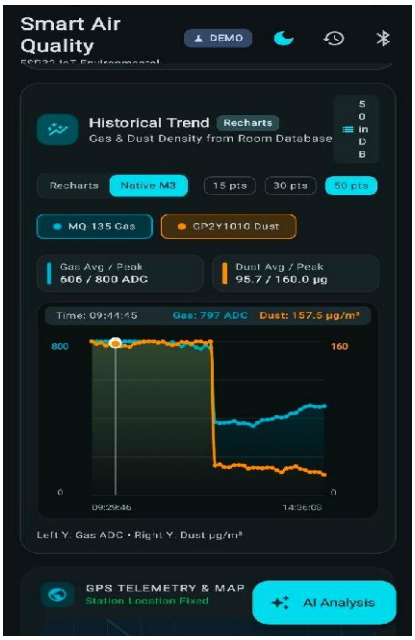


Figure 8.6: Gas and dust trend graph

8.1.7 Gemini AI Analysis

The analysis dialog shows a loading indicator and then a structured result: a summary, the safety status, sensor observations, health impact and recommendations. It is either the Gemini output or the offline analysis. The screenshot in Figure 8.7 was captured in demo mode and carries the footer “Generated by Smart AQI Rule Engine”, so it shows the offline rule-based result and not a Gemini response.



Figure 8.7: Gemini AI analysis dialog

8.1.8 Health / Environmental Recommendation

The recommendation part of the analysis lists practical actions such as ventilation, avoiding outdoor exertion or using a mask depending on the AQI band. It is general advice and not medical advice.



Figure 8.8: Health and environmental recommendations

8.1.9 Bluetooth Connection

The connection banner shows Disconnected, Connecting, Connected or Error, with buttons to connect or disconnect and a switch for demo mode. The device dialog lists paired Bluetooth devices so that the user can choose the ESP32.

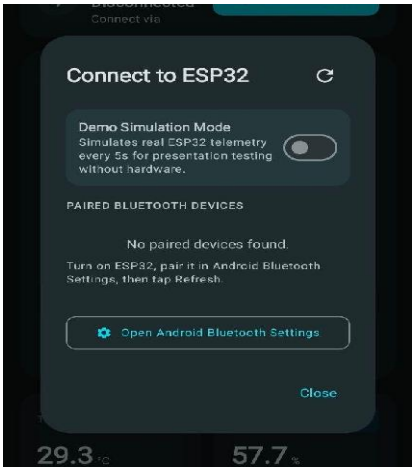


Figure 8.9: Bluetooth connection banner and device selection

8.1.10 Settings / Theme

The application has no separate settings screen. The only setting is the theme, changed with the top-bar button, which cycles System, Dark and Light. The choice is not saved between launches in the current code.

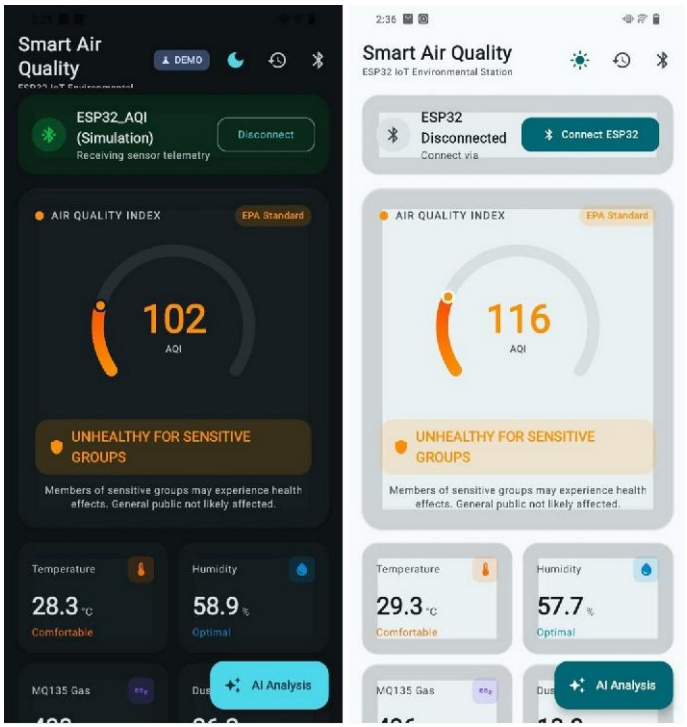


Figure 8.10: Theme (light and dark) display

8.2 Hardware Prototype

A photograph of the assembled station, showing the ESP32, the sensors, the GPS module, the battery and the converter on the breadboard, should be added here when the final prototype is photographed. The circuit diagram in Figure 5.3 shows the intended wiring.

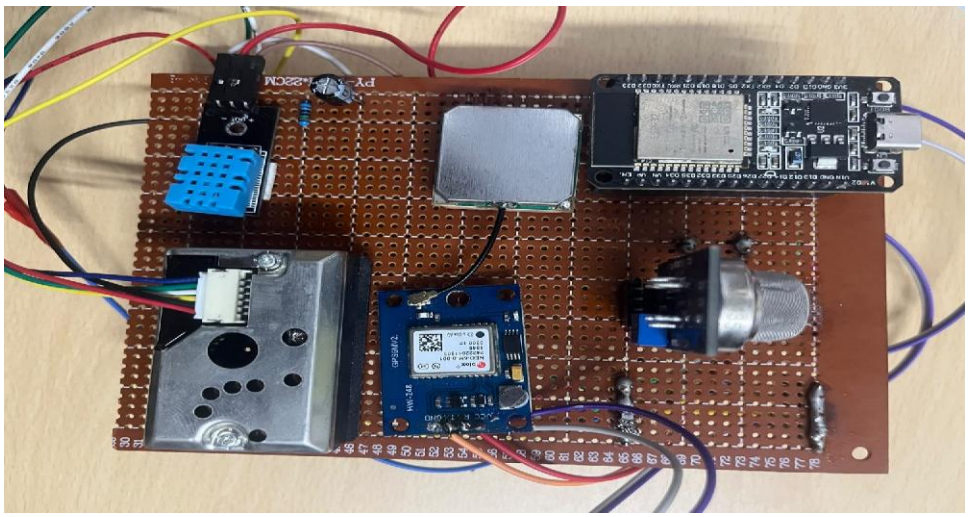


Figure 8.11: Photograph of the assembled hardware prototype

8.3 Sample Output Summary

The ten screens above cover the full functionality of the application. Before submission, the team should capture each screen on a phone connected to the real station, and also in demo mode, and insert the images in the marked places so that the figures can show actual readings. Where possible, the screenshots should be accompanied by the photograph of the assembled prototype.

CONCLUSION

This project designed and documented a Smart Air Quality Monitoring System using IoT and AI. An ESP32 station with DHT temperature and humidity, MQ-135 gas, GP2Y1010AU0F dust and NEO-6M GPS sensors computes a composite air-quality index and sends telemetry over Bluetooth Classic SPP to an Android application written in Kotlin with Jetpack Compose. The application shows the AQI, the environmental readings and the location, stores every reading in a Room database for history and trend charts, and offers a demo mode that runs without hardware. Gemini-based analysis explains the readings and gives recommendations, with a rule-based engine providing the same advice offline; it analyses current data and does not predict future air quality. Against the objectives of Chapter 1, the station design, telemetry link, Android application and AI analysis are documented or implemented, while the hardware verification and measurement campaign of Chapter 7 remain to be completed before the system can be called validated. The low-cost sensors are indicative and uncalibrated, so the index is a heuristic rather than a regulatory AQI, and no accuracy or reliability figures have been measured. Even so, the modular architecture makes the system a useful low-cost personal or classroom monitor that can be extended with further sensors and features.

FUTURE SCOPE:

The following improvements are proposed as future work; none is implemented in the present system.

- **Better sensing and calibration.** Add a laser particle sensor (PMS5003 or SDS011) and calibrate the sensors against a reference instrument.
- **Cloud synchronization and mapping.** Upload readings to the cloud and combine many stations into pollution maps.
- **Predictive AQI.** Train forecasting models on the stored history to predict short-term AQI.
- **Long-range connectivity.** Add LoRa or 4G so the station can report without a nearby phone.

REFERENCES

[1] S. G. Krishnan and W. N. H. Wan Kairuddin, "IoT indoor air quality monitoring system with Android app integration," Universiti Tun Hussein Onn Malaysia (UTHM) Publisher, published 28 Oct. 2025. [Online]. Available: https://publisher.uthm.edu.my/periodicals/index.php/eeee/article/view/20976

[2] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv:2312.11805, 2023.

[3] Y. A. U. Dilusha, T. Priyanka, M. Zahir and T. Yogamalar, "IoT-based air pollution monitoring system for surrounding environments in private education institute using MQ135 and MQ7 sensors," in Proc. International Conference on Business Innovation (ICOBI), NSBM Green University, Sri Lanka, 2022, pp. 438–444.

[4] World Health Organization, WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide, Geneva, 2021.

[5] V. A. Nguyen, V. H. Vu, V. S. Doan, T. H. Nguyen, D. P. Thuan, K. Nguyen, P. L. Nguyen and M. T. Le, "Realizing mobile air quality monitoring system: architectural concept and device prototype," in Proc. Asia-Pacific Conference on Communications (APCC), 2021.

[6] S. Kaivonen and E. C.-H. Ngai, "Real-time air pollution monitoring with sensors on city bus," Digital Communications and Networks, vol. 6, no. 1, pp. 23–30, 2020, doi: 10.1016/j.dcan.2019.03.003.

[7] M. Mokrani et al., "Air quality monitoring using IoT: a survey," in Proc. IEEE International Conference on Smart Internet of Things (SmartIoT), 2019, doi: 10.1109/SmartIoT.2019.00028.

[8] S. Dhingra, R. B. Madda, A. H. Gandomi, R. Patan and M. Daneshmand, "Internet of Things mobile–air pollution monitoring system (IoT-Mobair)," IEEE Internet of Things Journal, vol. 6, no. 3, pp. 5577–5584, 2019.

[9] U.S. Environmental Protection Agency, Technical Assistance Document for the Reporting of Daily Air Quality – the Air Quality Index (AQI), EPA-454/B-18-007, 2018.

[10] L. Morawska et al., "Applications of low-cost sensing technologies for air quality monitoring and exposure assessment: How far have they gone?," Environment International, vol. 116, pp. 286–299, 2018, doi: 10.1016/j.envint.2018.04.018.

[11] N. Castell et al., "Can commercial low-cost sensor platforms contribute to air quality monitoring and exposure estimates?," Environment International, vol. 99, pp. 293–302, 2017.

An IoT-Enabled Multisensor Air Quality Monitoring Prototype with Edge-Based Composite Air Quality Score Computation and On-Demand AI-Assisted Mobile Interpretation

ABSTRACT - ***Low-cost air quality monitors on microcontrollers often just show the sensor numbers and stop there. The user has to figure out what it means. This prototype puts together an ESP32 board with a few sensors like the DHT11 for temperature and humidity, an MQ 135 gas sensor, a dust sensor, and a GPS module. There is also a mobile app made with Flutter.***

The board works out a score called CAQS by combining the dust reading, the gas sensor value, and temperature with some weights the team picked. It turns the score into one of six labels and sends everything as simple JSON over Bluetooth. No internet or cloud stuff is involved. The score is not an official air quality index because the inputs are not calibrated and the weights are just internal choices.

The app can also send the readings to Google Gemini when asked and show a plain language note about them. That part is only for help with understanding and not a real prediction tool. Tests in normal conditions and with extra particles showed the data format worked fine over both serial and Bluetooth. The category moved from good at a low score to hazardous at a high one.

The dust number came out way too high for indoor air though and it pretty much decided the whole score. It is near the top of what the sensor can report. No checks against reference tools or measurements of power use or range were done so this just shows the setup can run and not that the numbers are accurate. It feels like the dust part might need more work but that was not tested here.

KEYWORDS – ESP32, Air Quality Monitoring, Composite Air Quality Score, Bluetooth Classic, Flutter, Generative AI, Environmental Monitoring, GPS

I. INTRODUCTION

Low cost IoT nodes have been reported for this kind of thing. It seems continuous local environmental monitoring is becoming an engineering concern these days. Air quality affects human health comfort and productivity. There is one with a Raspberry Pi that acts as a web node for checking air. I think that part helps, but I am not totally sure how the whole setup works in practice. [1], an ESP32 that reads temperature and humidity [2]. An ESP32 node that uses a DHT11, plus an MQ-7 and an MQ-135 [3]. And there is also an ESP32 weather station [4]. Air temperature, moisture level, dust particles, and gas amounts change over time and across different places. Because of that, readings from one main weather or air station far away might not match what is happening in a specific room, a bus or car, or a nearby street. [4]. This creates a need for inexpensive sensors that monitor the immediate surroundings.Existing technologies that measure local environment awareness are based on either manual gauging or single-purpose analog gauges. Both of these systems are unable to perform continuous multifactor measurements and offer little or no analysis of the results when an anomaly occurs. Using an electronic device that displays information on a minuscule screen is no better than the analog gauges because it does not explain what the deviations mean.Falling prices of system-on-chip microcontrollers with built-in wireless interfaces have made hobbyist-level monitoring nodes affordable. An ESP32 microcontroller plus a DHT22 temperature-humidity sensor was used to report temperature and humidity data to the Blynk platform. [2], a development board of ESP32 with DHT11, MQ-7, MQ-135, that has driven OLED and LED displays [3], and an ESP32-WROOM DEVKIT station integrated BME280 weather sensor, MQ-135, and ML8511 UV sensor [4]. Together with cloud services such as ThingSpeak, Blynk, or Google Sheets, such nodes allow for real-time monitoring at a low cost, as in the Raspberry Pi node, which used ThingSpeak via esp32 and sensors. [1].

However, the weakness inherent to these systems is that they stop at the data collection and presentation stage. They either show the raw sensor values or a threshold-based colorization. None of the systems reviewed in Section II offer any natural-language interpretation of the observed state that might assist the end-user in responding to the observed situation.Narrowing this functionality gap does not necessarily require a machine learning model that would run on the device. A separate application could encode the sensor state as a narrowly-framed natural language question and report it, on demand, to a cloud-based generative model that would provide an appropriate context-based reply without requiring a separate classifier trained on domain-specific data, so long the exchanged messages are framed in a way that prevents the model from making up information or attempting to diagnose the cause of the observations. Note that such an approach would not be guaranteed to provide accurate information; therefore, the exchange should be framed as an explanation of the sensor values rather than an assertion of objective truth.

In this paper we describe a prototype air quality monitoring system with three components: (i) an ESP32 sensing node with a DHT11, an MQ-135 analog gas sensor, a GP2Y1010 optical dust sensor, and a NEO-6M GPS module, which computes a prototype-specific Composite Air Quality Score (CAQS) locally, then sends the readings, score, category, and GPS fields as newline-delimited JSON over Bluetooth Classic SPP; (ii) a cross-platform Flutter/Dart application that parses the stream and displays it on a live

dashboard; and (iii) an on-demand function in which the application sends the current readings to the Google Gemini API and displays a natural-language explanation of the results. The contributions are (1) a documented ESP32-to-phone Bluetooth pipeline whose JSON payload was sanity-checked against a USB-serial log; (2) an explicit, reproducible description of the CAQS and sensor-processing equations and their limitations; (3) an application design that combines a live visualization, GPS status, and on-demand AI interpretation; and (4) a functional evaluation under a baseline and a particulate-increased condition, with an explicit discussion of what the data do and do not show. We make no claim of regulatory accuracy, calibration, or forecasting performance.

II. LITERATURE SURVEY

All environmental monitoring systems described in the scientific literature have similar concepts: the microcontroller or single board computer that is connected to a low-cost sensor suite, wireless communication to send data, and either cloud computing or widgets for storing and displaying data. The variations among these systems are due to the selection of the processing platform, sensor suite, communication method to leave the node, finally, whether any sort of data interpretation has been carried out before the data reaching the end user. Single-board computers offer more processing power than microcontrollers and have been used as sensor nodes for this reason, as described next.Balasubramaniyan and Manivannan [1] built an IoT node based on a Raspberry Pi processor using a GrovePi+ shield and equipped with sensors for evaluating humidity, dust, MQ-5, MQ-7, and MQ-135 gas concentrations. The data was uploaded to the cloud via WiFi and hosted on ThingSpeak where MATLAB analytic software was used to process the information and post alerts to Twitter. The processing power was offloaded to the cloud to reduce the load on the IoT node. However, the system only supported four nodes, was based on the HTTP protocol, and did not have a client application.Many systems based on the ESP32 are low-cost solutions that rely on the ESP32's Wi-Fi to connect to the cloud or a widget platform. Adeagbo [2] used an ESP32 and a DHT22 sensor connected to the Blynk platform to indicate the temperature and humidity levels in his local area using a mobile widget. The result indicated that while the temperature value reported was 6 o C, the actual value was 4 o C (20% error), whereas the humidity level had an error of 2%. Omkar et al. [3] used an ESP32 microcontroller and DHT11, MQ-7, and MQ-135 sensors in an air quality monitoring system that used an OLED display, LED indicators, and Blynk or ThingSpeak platform but without giving quantitative results on a comparative basis. On the other hand, Uddin and Praharaj [4] The existing environment monitoring solutions designed for personal or small business use employ a common set of features. The first solution combined a BME280, an MQ-135, an ML8511 UV sensor, and a rain sensor on an ESP32-WROOM DEVKIT that logged to Google Sheets, served an HTML dashboard, and mirrored data to Blynk; their e-mail alert feature was not fully validated. The three main limitations of such solutions are (i) the requirement of a Wi-Fi link to a cloud or widget platform, (ii) a user interface implemented as a third-party widget service, a web page, or an LED/OLED display, and (iii) the absence of a natural-language explanation of the combined sensor state beyond fixed threshold checks..

The most recent related work [5] used an ESP32-WROOM node with DHT22, BH1750, and DSM501A sensors and MQTT over Wi-Fi to ThingSpeak. It is reported there, besides the calibration procedure against reference instruments, statistical error metrics, and measurements of latency, packet delivery, and power. However, its comparison with commercial monitors was based on datasheet specifications and not on parallel measurements [5]. This work provides what a quantitative evaluation of such a node entails: a reference-instrument comparison, calibration, repeated measurements, and network and power characterization. The prototype under evaluation here has not been subjected to these tests, and its results cannot be compared to the numbers reported in [5].The reviews of the field indicate the same pattern. IoT-based air quality monitoring systems usually consist of multiple sensors, a microcontroller, and a communicating module, the data from which are displayed on a monitoring interface [6]. In a survey of low-cost IoT pollution monitors, ESP-series microcontrollers were used in approximately 40% of the systems, and IoT cloud and web services were the preferred interface in approximately 53% [7]. Another review of low-cost indoor systems identified Wi-Fi as the prevalent communication technology and pointed to sensor calibration and energy consumption as common challenges associated with accuracy and usability [8]. A direct Bluetooth-to-phone design thus departs from the norm of the Wi-Fi and cloud pattern, and calibration is a recognized challenge of low-cost systems that the current prototype has not yet tackled.

III. PROPOSED METHODOLOGY

The system is implemented as an end-to-end pipeline. Environmental sensors are read by an ESP32 acquisition node that computes a prototype-specific composite score locally, then serializes it as JSON over a direct Bluetooth Classic serial connection and streams it up to paired smartphones; a Flutter app parses the JSON into a live dashboard. At the user's request, the Flutter application issues the current readings to a cloud-hosted generative-AI model that replies with an explanatory text. The individual pipeline stage descriptions are provided in the subsequent subsections.

A. Overall System Architecture

The system features five stages that are illustrated in Fig. 1. The sensor node consists of an ESP32-WROOM-32 development board interconnected on a breadboard with the DHT11, MQ-135, GP2Y1010, and NEO-6M modules (Section III-B); the GPS module is part of the sensor node and its fields are sent in the same payload. The ESP32 reads each sensor and calculates the Composite Air Quality Score (CAQS) and its category locally (Section III-C) and serializes the result as JSON. A Bluetooth Classic Serial Port Profile (SPP) link carries the payload to a paired Android device that is devoid of Wi-Fi or a cloud broker. The Flutter application receives and parses the stream and displays the readings, the CAQS, and the GPS state. Only on the explicit request of the user, does the application send one HTTPS request to the Google Gemini API whose response is displayed as an environmental explanation (Section III-F). The Gemini model does not participate in the CAQS calculation which is only made on the ESP32.

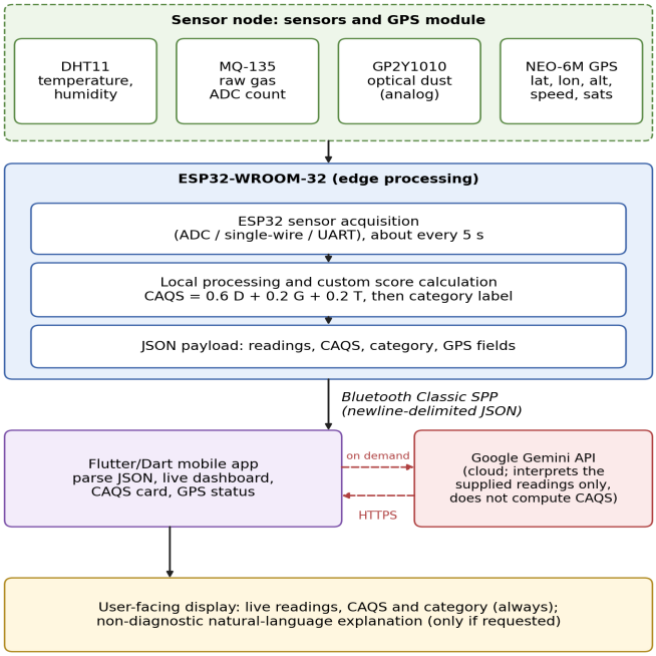


Fig. 1: Overall system architecture and data flow of the prototype

B. Environmental Sensing Unit

The sensing unit consists of four modules connected to the ESP32 (Table I). DHT11 is a low-cost digital sensor that provides temperature and relative humidity over a single-wire interface; its resolution and accuracy are limited. MQ-135 is an analog metal-oxide gas sensor module that responds to several gases, for example ammonia, nitrogen oxides, benzene, smoke, and carbon dioxide, but it can not distinguish between them [3]. Its analog output is read by the 12-bit ESP32 ADC as a raw count from 0 to 4095. This count should not be interpreted as a calibrated concentration; thus, the gas channel is used only as a relative indication and is never expressed in ppm. The GP2Y1010 is an infrared optical dust sensor, whose analog output is converted by the firmware to a voltage and then to a dust-density estimate in µg/m³ using Eqs. (1) and (2); that is, this estimate is not a calibrated PM2.5 or PM10 value (Section III-C). Low-cost light-scattering particulate sensors, including the GP2Y1010AU0F, are not inherently calibrated and must be evaluated against reference instruments before use [9],

The GP2Y1010AU0F works in a pulsed mode where its output voltage is to be sampled at a specific moment of each pulse [10]. The NEO-6M GPS module outputs latitude, longitude, altitude, ground speed, and satellite count over a UART, so that a reading can be geotagged if a satellite fix exists. The GPS module is part of the sensor node, and its fields are transmitted in the same JSON payload. Since these parts are low-cost, none of the gas, dust, temperature, or humidity channels is claimed to have laboratory-grade accuracy, and the score defined in Section III-C is not a certified regulatory index.

TABLE I. SENSOR AND MODULE INTERFACES OF THE PROTOTYPE

Module	Quantity	Interface and firmware use
DHT11	Temperature, relative humidity	Single-wire digital input; values used directly
MQ-135	Relative gas indication	Analog output to 12-bit ESP32 ADC; raw count 0–4095 used as G
GP2Y1010	Optical dust sensor	Analog output to ESP32 ADC; converted to V_D and dust estimate D (Eqs. 1–2)
NEO-6M	Latitude, longitude, altitude, speed, satellites	UART; fields sent in the JSON payload (zero until a fix)

C. ESP32-Based Data Acquisition and Composite Air Quality Score

The firmware reads the sensors approximately every five seconds: it samples the DHT11, reads the MQ-135 and GP2Y1010 analog channels through the ESP32 ADC, and reads the NEO-6M over UART. From these inputs it computes the dust estimate, the CAQS, and a category string (Eqs. (1)–(3)) and packs every reading set into a single-line JSON object with the fields temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, and satellites. The key names aqi and airQuality are retained from the firmware and application code, but they carry the CAQS and its category label. The object is written to the USB serial port at 115200 baud and to the Bluetooth Classic SPP serial interface; the two outputs carried the same payload during testing (Section IV). The payload contains no timestamp field. The GPS fields remain zero until a satellite fix is obtained, which did not happen during the indoor tests (Section IV).

Dust estimate. The GP2Y1010 analog output is converted to a voltage by Eq. (1), where ADCD is the 12-bit ADC count (0 to 4095) and 3.3 V is taken as the ADC full-scale voltage. The dust-density estimate D in µg/m³ is then calculated by Eq. (2):

$$VD = ADCD \times (3.3 / 4095) \quad \text{----->} \quad (1)$$

$$D = \max(0, (VD - 0.1) \times 1000) \quad \text{----->} \quad (2)$$

where VD is in volts, 0.1 V is a fixed offset, and 1000 µg/m³ per volt is a fixed slope in the firmware. These constants constitute an uncalibrated linearization; they were not obtained by comparison with a reference particle instrument, and the outcome is therefore not a measured concentration of PM2.5 or PM10. Since the ADC range is 0 to 3.3 V, D cannot exceed $(3.3 - 0.1) \times 1000 = 3200$ µg/m³, and ADC nonlinearity at full scale may additionally distort the high values.Gas inputs. The gas value G in Eq. (3) is simply the raw MQ-135 ADC count (0 to 4095); it was not converted to sensor resistor, concentration, or ppm, but was used directly. The count depends on sensor warm-up time, temperature, and humidity, as well as on the gas mixture. A calibration against a reference gas was not done in this work.Composite Air Quality Score. The prototype computes the dimensionless score S of Eq. (3), where T is the DHT11 temperature in °C:

$$S = 0.6\,D + 0.2\,G + 0.2\,T \quad \text{---> (3)}$$

The weights 0.6, 0.2 and 0.2 are internal design choices of the prototype; they were not determined from a validated health- or regulatory-based methodology. The three inputs have different units and ranges (µg/m³-based estimate, unitless ADC count, and °C) and are not normalized. The score is reported as an integer (the reported values are consistent with truncation) and mapped to a category label, evaluated in this order: GOOD ($S \leq 50$), MODERATE ($S \leq$

100), UNHEALTHY FOR SENSITIVE GROUPS ($S \leq 150$), UNHEALTHY ($S \leq 200$), VERY UNHEALTHY ($S \leq 300$), and HAZARDOUS ($S > 300$). These labels and thresholds are reminiscent of the category names of regulatory air quality indices, but S is not a regulatory AQI: it is not calculated from pollutant concentrations over averaging periods using official breakpoints, and a CAQS category does not imply the corresponding regulatory category or health effect. For this reason the output is called the CAQS in this paper, although the JSON field name is aqi. Because T is added directly, a warm environment raises S even if the air is clean, and the large ranges of D and G compared with T mean that dust and gas dominate S in practice.Gas and optical dust sensors are sensitive towards temperature, humidity, airflow, and aging, and they require warm-up and calibration in relation to a reference instrument in order for their readings to be interpreted quantitatively. A such comparison was not performed in this work (Section V).

D. Device-to-Application Communication (Bluetooth)

Rather than a Wi-Fi uplink, the ESP32 is acting as a Bluetooth Classic SPP device announced as "ESP32_AQI." On the application side, the flutter_bluetooth_serial_plus package is employed: it initially checks Bluetooth is enabled, then scans the list of bonded devices for "ESP32_AQI" and opens connection using its address. It follows that the module must be paired in advance from the phone's Bluetooth settings. Once connected, the app decodes each incoming chunk as UTF-8 and appending to an internal buffer, and any time the buffer has a newline character, it extracts the text up to the newline, trims it, and passes it to a JSON decoder, and the remainder keeps in the buffer. This buffering is needed because SPP delivers an arbitrary byte stream, not message-framed packets, so one JSON object may arrive split across several callbacks. The only internet access in the system is initiated by the phone application, on demand, when the user requests the AI analysis (Section III-F). In this implementation the ESP32 does not use Wi-Fi and has no direct internet or cloud connection.

E. Flutter Application

The Dashboard is the single-page Flutter application written in Dart with the same code base potentially targeting Android, Windows, and Linux, although the results in Section IV were obtained on Android. It deliberately keeps the architecture simple and focuses on a single stateful widget: each incoming Bluetooth message is parsed into fields of typed data (temperature, humidity, gas, dust, aqi, airQuality, latitude, longitude, altitude, speed, satellites) stored within the widget state, and each successful parsing refreshes the interface by Flutter's setState() mechanism. The interface includes a Bluetooth connection card showing the device name and connection status, a large score card indicating the CAQS and its category with corresponding colors (green, orange, deep orange, red, purple, and brown) across six severity bands, separate cards for temperature, humidity, gas, and dust, a GPS information card, and an AI-analysis button with a result panel. In the screenshot of Fig. 2 the score card is labeled "Air Quality Index" indicating the CAQS defined in Section III-C. The evaluated version does not store the past readings over sessions and does not plot the time-series charts, so each screen only reflects the most recent parsed reading. A later native Android re-implementation (using Kotlin with Jetpack Compose) includes the local history storage and trend charts, but was not part of the tests reported here.

TABLE II SOFTWARE PACKAGES USED IN THE FLUTTER APPLICATION

Component	Package	Purpose
Bluetooth link	flutter_bluetooth_serial_plus 0.5.x	Discovers the paired ESP32_AQI device, opens a Bluetooth Classic SPP connection, and streams the JSON data.
AI HTTP client	http 1.6.x	Sends the constrained prompt to the Gemini generateContent REST endpoint and parses the response.

F. Environmental Analysis Using AI (Gemini)

The application decouples deterministic and generative functions. The deterministic function is the CAQS and category computed by the ESP32 firmware (Section III-C); the application only displays it, and it is available at any time when the Bluetooth link is up, without internet access or an AI service. The generative function is on-demand, AI-assisted interpretation: when the user presses the "AI Analyze" button, which is only available while connected, the application constructs a text prompt containing the current temperature, humidity, gas ADC value, dust estimate, CAQS and category, and GPS status, and sends it over HTTPS to the Google Gemini generateContent endpoint. The prompt requires the response to use fixed headings (Overall Assessment; Risk Level, which is LOW, MODERATE, HIGH, or CRITICAL; Key Observations; Possible Causes; Precautions; Health Considerations; Environmental Recommendations; Short-Term Prediction; and Important Warning) and instructs the model to use only the values supplied, to treat the CAQS as a custom score and not a regulatory index, to treat the gas value as an ADC reading and not a concentration, and not to provide a medical diagnosis. The response text is displayed verbatim in a result card. The Gemini model does not calculate the sensor score; this is done only on the ESP32. The "Short-Term Prediction" section is text generated by the language model from a single set of readings; it is not produced by a validated time-series forecasting model and must not be interpreted as a pollution forecast. The risk level and the health-related statements in the response are likewise text generated and not validated assessments, and there is no claim that they are accurate or consistent.

G. Data Processing and Decision Flow

The end-to-end process takes the following form: (1) the ESP32 microcontroller reads the DHT11, gas, dust, and GPS sensors; (2) the firmware calculates the dust estimate, CAQS, and category string (Eq. (1)-(3)); (3) the measurements are formatted into a JSON single-line object; (4) the object is pushed through the USB serial interface and Bluetooth Classic SPP; (5) the input-stream listener in the Flutter application gathers the incoming bytes until a new line is detected; (6) the message is parsed as JSON, and the sensor, CAQS, and GPS fields are updated; (7) the interface is rebuilt, and the cards are re-painted; (8) when the user presses "AI Analyze" while being connected, the current

prompt is formed as per the description of the constrained prompt of Section III-F; (9) the prompt is sent to the Gemini API over HTTPS, and the text response is extracted; and (10) the response text is placed in the result card. The steps (1)-(7) are repeated as long as Bluetooth messages arrive at the approximate rate of once every five seconds, whereas steps (8)-(10) are taken upon user request.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

The prototype was only tested at a functional level because there was no certified reference instrument available for calibration/comparison. The results below are therefore direct observations of the ESP32 output, the Flutter application, and the Gemini response; they demonstrate feasibility but do not establish measurement accuracy. Table III lists readings from two test conditions recorded with the same firmware version. The first is an example of a low-pollutant indoor baseline, read from the Arduino IDE serial monitor and compared against the Bluetooth JSON payload; the gas ADC reading was 0, the dust estimate was 0, and the firmware reported a CAQS of 5 (GOOD). The second was recorded in the Flutter application after exposure to a higher-particulate environment; the gas ADC reading was 186, the dust estimate was about 2947–2967 µg/m³, and the reported CAQS was 1823 (HAZARDOUS). Neither test was performed in a controlled manner. Only representative values are reported for each condition and the number of samples is not stated, so means and standard deviations are not specified and the values cannot be used for calibration. The reported scores correspond with Eq. (3): for the baseline, $0.2 \times 28.9 = 5.78$, reported as 5; for the elevated condition, the application screenshot (Fig. 2) shows a dust estimate of 2967.91 µg/m³, and $0.6 \times 2967.91 + 0.2 \times 186 + 0.2 \times 29.3 = 1823.8$, reported as 1823. The category therefore changed in the expected direction with changes in the inputs, and the new category was displayed correctly in the application.

TABLE III. REPRESENTATIVE SENSOR READINGS UNDER TWO TEST CONDITIONS

Parameter	Baseline (Indoor, Serial Log)	Elevated Condition (App Test)
Temperature (°C)	28.90	29.3
Humidity (%)	70.00	70.0
Gas (raw ADC)	0	186
Dust estimate (µg/m³)	0.00	~2947–2967
CAQS (on-device score)	5	1823
CAQS category	GOOD	HAZARDOUS
GPS satellites / fix	0 / no fix	0 / no fix

Plausibility of the dust estimate. A dust estimate of about 2950 µg/m³ is orders of magnitude above values typical of indoor air and is not plausible as a PM2.5 or PM10 concentration. Under the Eqs. (1) and (2) the largest possible estimate is 3200 µg/m³, reached at the ADC full-scale voltage, and a value of 2967.91 µg/m³ corresponds to VD ≈ 3.07 V, about 93% of full scale. The reading is thus close to the upper bound of the conversion range and is compatible with a saturated or poorly scaled analog signal (for example from the sensor output range, ADC attenuation or nonlinearity, or the sensor sampling), and not with a physical measured concentration. The cause was not investigated in this work. Because the dust term has weight 0.6, it contributed $0.6 \times 2967.91 \approx 1780.7$ of the total of about 1823.8 (about 97.6%), while the gas term contributed 37.2 and the temperature term about 5.9. The elevated-condition CAQS is thus almost entirely determined by the dust estimate, and its numerical value should not be interpreted as a severity level. A gas reading of 0 in the baseline condition is also unusual for a powered MQ-135 module and may indicate insufficient warm-up or an input-range problem; this was likewise not investigated.During the test sessions, newline-based framing in the application's stream buffer recovered complete JSON objects, and the payload received over Bluetooth matched the payload shown on the USB serial monitor at the same time. This is a functional check of message framing only: packet delivery ratio, latency, throughput, and communication range were not measured. The GPS module obtained no satellite fix in either (indoor) test session; the application stayed in the "Waiting for satellites…" state, and the latitude, longitude, altitude, and speed fields were parsed as zero. The absence of a fix is attributed to the indoor test environment, but this attribution was not verified outdoors, and GPS accuracy was not evaluated.



Fig. 2: Application screenshot during the elevated-particulate test (CAQS 1823; dust estimate 2967.91 µg/m³). The on-screen label "Air Quality Index" denotes the CAQS.

The on-demand AI analysis was exercised with the Gemini API while the application was connected to the sensor node. Because the output is free text and not a numeric score, no accuracy percentage can be given. In the sessions performed, the responses followed the required headings, used one of the four risk levels, and described the score as a custom score and the gas value as an ADC reading, as the prompt instructs. This is a qualitative observation of format

compliance; the number of responses examined is not reported here, and the observation is not an evaluation of correctness, consistency across repeated requests with identical inputs, or forecasting ability.

V. LIMITATIONS AND FUTURE EXPERIMENTS

Measurement validity. The gas input is a raw ADC count, not a concentration; the dust value is an uncalibrated voltage-based estimate, and neither has been compared to a reference instrument. The sensors are sensitive to warm-up, temperature, humidity, airflow, and age, and the DHT11 is a low-cost sensor with limited resolution and accuracy. Correct operation of the optical dust sensor also depends on its LED pulse timing and sampling which were not independently verified [10], and the elevated dust estimate is near the upper limit of its conversion range and physically unreasonable.Score validity. The CAQS weights are internal design options without a validated derivation; the inputs are not normalized; temperature is included as an additive term; and category thresholds are applied to a dimensionless score. The CAQS is therefore not equivalent to a regulatory AQI and should not be used for health or compliance purposes.Evaluation scope. Only two uncontrolled conditions were recorded; no repeated measures, reference monitor, or statistical analysis was made. Latency, packet delivery, Bluetooth range, power consumption, battery life, GPS accuracy, and AI-response correctness were not measured. The Bluetooth architecture requires pairing and short range, and provides no remote access; the tested application version stores no history; the AI function needs an internet connection on the phone, and a third-party service; and testing was only on Android. Planned experiments. The following experiments are planned and have not been carried out: (1) repeated measurements under controlled conditions, (2) baseline, moderate-change and elevated-particulate conditions, (3) outdoor GPS validation with a valid fix, (4) Bluetooth tests at a number of measured distances, (5) packet-loss and latency evaluation, (6) power and current measurements, (7) comparison with a suitable reference monitor from which MAE and RMSE can then be computed, and (8) tests of repeatability and of temperature and humidity influence.

VI. CONCLUSION AND FUTURE WORK

This paper introduced a prototype Bluetooth-based ESP32 air quality monitor, which combined a DHT11, an MQ-135 gas sensor, a GP2Y1010 dust sensor, a NEO-6M GPS module, and a Flutter application with optional on-demand AI-assisted interpretation via the Gemini API. The functioning was demonstrated: the JSON payload was transferred over USB serial and Bluetooth Classic SPP, the application's newline-based buffering parsed the messages, and the CAQS and its category changed in the expected direction from a baseline (5, GOOD) to a particulate-increased condition (1823, HAZARDOUS). The Gemini function output structured, non-diagnostic text in the tested sessions; its correctness and consistency were not evaluated. These results demonstrate the functional viability of a direct device-to-phone architecture and not measurement accuracy: the gas and dust channels are uncalibrated, the elevated dust estimate is implausible and near the conversion limit, and the CAQS is a prototype-specific score dominated by the dust term. Future work will calibrate the sensors against a reference monitor, redesign the score from normalized, pollutant-based inputs, carry out the experiments listed in Section V, extend the application with history storage and trend charts (a native Android re-implementation with these functions has been developed but was not part of the tests reported here), and evaluate the AI responses formally.

REFERENCES

[1] C. Balasubramaniyan and D. Manivannan, "IoT Enabled Air Quality Monitoring System (AQMS) using Raspberry Pi," Indian J. Sci. Technol ., vol. 9, no. 39, pp. 1–5, Oct. 2016. DOI: 10.17485/ijst/2016/v9i39/90414.

[2] A. A. Adeagbo, "IoT based environment monitoring system using ESP32," arXiv:2405.14047, May 2024. DOI: 10.48550/arXiv.2405.14047.

[3] N. Omkar, S. Rahul, R. Vijaykrishna, N. Swapna, and M. R. Sura, "Smart environmental monitoring using ESP32 microcontroller," IOSR J. Electron. Commun. Eng., vol. 19, no. 3, ser. 1, pp. 07–12, May-Jun. 2024, doi: 10.9790/2834-1903010712.

[4] M. S. Uddin and M. K. Praharaj, "IoT based weather monitoring system using ESP32 microcontroller," Int. J. Adv. Res. Sci. Commun. Technol., vol. 5, no. 2, pp. 693–701, Oct. 2025, doi: 10.48175/IJARSCT-29283.

[5] M. S. Bakare and K. Abubaker, "IoT-based indoor environmental monitoring system using multi-parameter sensing and ESP32-WROOM integration," Discover Electron., vol. 3, Art. no. 6, 2026, doi: 10.1007/s44291-026-00157-3.

[6] J. Saini, M. Dutta, and G. Marques, "Indoor air quality monitoring systems based on Internet of Things: A systematic review," Int. J. Environ. Res. Public Health, vol. 17, no. 14, Art. no. 4942, July 2020, doi: 10.3390/ijerph17144942.

[7] O. Alsamrai, M. D. Redel-Macías, S. Pinzi, and M. P. Dorado, "A systematic review for indoor and outdoor air pollution monitoring systems based on Internet of Things," Sustainability, vol. 16, no. 11, Art. no. 4353, 2024, doi: 10.3390/su16114353.

[8] J. Peixe and G. Marques, "Low-cost IoT-enabled indoor air quality monitoring systems: A systematic review," J. Ambient Intell. Smart Environ., vol. 16, no. 2, pp. 167–180, 2024, doi: 10.3233/AIS-220577.

[9] M. Ghamari, H. Kamangir, K. Arezoo, and K. Alipour, "Evaluation and calibration of low-cost off-the-shelf particulate matter sensors using machine learning techniques," IET Wirel. Sens. Syst., vol. 12, no. 5–6, pp. 134–148, 2022, doi: 10.1049/wss2.12043.

[10] K. Bučar, J. Malet, L. Stabile, J. Pražnikar, S. Seeger, and M. Žitnik, "Statistics of a Sharp GP2Y low-cost aerosol PM sensor output signals," Sensors, vol. 20, no. 23, Art. no. 6707, Nov. 2020, doi: 10.3390/s20236707.